Cutoff — Himalayan Flood, Debris & Access Mapping from Space

Consolidated research and implementation blueprint · Multimodal AI Hackathon 2026 · Track B “Mapping Flood Damage from Space”

11 October 2026 · version 1.0

Contents

1. Executive summary
2. Beginner-friendly overview
3. Official problem statement and requirements
    3.1 Source of authority
    3.2 The problem, users, and why it matters
    3.3 Requirements matrix (official vs ours)
    3.4 Inputs, outputs, constraints, scope
4. Research methodology and source inventory
    4.1 Method
    4.2 Source inventory
5. Existing systems and literature review
    5.1 Research papers and datasets
    5.2 Repositories (code actually present vs claims)
    5.3 Operational systems
    5.4 What the landscape tells us
6. Research gap and defensible novelty
    6.1 Gap analysis
    6.2 Comparison matrix
    6.3 Minimum set of differentiators
7. Final problem definition and objectives
    7.1 Project specification
    7.2 Three explanations
    7.3 Conflict-resolution table
    7.4 MVP and optional features
    7.5 Research questions and hypotheses (all [H] until measured)
8. Satellite technology fundamentals
    8.1 Remote sensing in one paragraph
    8.2 Sentinel-1 and SAR
    8.3 Sentinel-2 and optical imagery
    8.4 Why SAR and optical are complementary — and why they can’t simply be stacked
    8.5 Task vocabulary (not interchangeable)
9. Dataset comparison and selection
    9.1 Candidate datasets
    9.2 Final dataset strategy
    9.3 Label harmonisation
10. Data preprocessing
    10.1 Why preprocessing must mirror Kuro Siwo
    10.2 Step-by-step pipeline
    10.3 Handling missing or unsuitable Sentinel-2
    10.4 Live-judging preprocessing modes [D]
11. Model comparison and final selection
    11.1 Candidates
    11.2 Final decision [D]
    11.3 Missing modalities
    11.4 Hazard product (model + rules + gating)
    11.5 Baselines
    11.6 Optional optical ablation (E9b, stretch)
12. Training methodology
    12.1 Starting configuration (to be tuned; not results)
    12.2 Staged plan
    12.3 CPU-only vs GPU plan
13. Evaluation strategy
    13.1 Metrics in plain language
    13.2 Protocol
    13.3 Experiments
    13.4 Results templates (fill after experiments — no numbers yet)
    13.5 Interpretation rules

14. System architecture
    14.1 System architecture
    14.2 Model-level architecture
    14.3 Data flow and artefacts
    14.4 Inference sequence
    14.5 Walk-through of a real user scenario
    14.6 Error handling and fallbacks
15. Technology stack
16. Repository structure
    16.1 File responsibilities
17. Detailed implementation plan
18. Code-level design and core examples
    18.1 SAR radiometry shared by training and inference — `geo/sar.py`
    18.2 Terrain — `geo/terrain.py`
    18.3 Dataset class — `data/kuro_siwo.py`
    18.4 Model, checkpoint save/load — `ml/model.py`
    18.5 Loss — `ml/losses.py`
    18.6 Training loop — `ml/train.py` and calibration — `ml/calibrate.py`
    18.7 Metrics — `ml/metrics.py`
    18.8 Tiled inference on an unseen image — `ml/infer.py`
    18.9 Rule-based disturbance, gating and optical evidence
    18.10 Georeferenced outputs — `geo/rasterio_io.py`
    18.11 Same-track pairing and OSM snapshot — `data/s1_pairs.py`, `data/osm.py`
    18.12 Road network and cut-off status — `geo/network.py`
    18.13 Results schema and situation report — `report/`
    18.14 Serving and displaying a prediction — `app/streamlit_app.py`
    18.15 Check-only evaluation — `eval/emsr927.py`
    18.16 Tests
19. Dashboard and demonstration workflow
    19.1 Layout
    19.2 Interactions, loading and error states
    19.3 Situation report (one page)
    19.4 Demo data needed
    19.5 3-minute video script
20. Testing and reliability
21. Compute and cost analysis
22. Hackathon execution plan
    22.1 Judge’s-eye matrix
    22.2 Recommendations
    22.3 Questions for the organisers (ask on day 1)
23. Risks and limitations
    23.1 Risk register
    23.2 Limitations to state in the report
24. Future improvements
25. Final conclusions
26. References
27. Appendices
    27.1 `configs/default.yaml` (template)
    27.2 `configs/kuro_siwo_splits.yaml` (from the repo’s `data_config.json` [V])
    27.3 Experiment tracking and reproducibility
    27.4 Prioritised checklist
    27.5 Final validation audit

> **Prepared for:** the Cutoff hackathon team (up to 4 members, 15-day build) **Scope:** official problem statement → research synthesis → final design → code-level blueprint → audit **Status of numbers in this document:** no experiment has been run yet. Every accuracy figure we quote is either (a) a published result from someone else, with its source and context, or (b) a *target* we set in advance and label **[TARGET]**. Nothing here is a measured result of our system.

Evidence labels used throughout

Label	Meaning
[V]	Verified on 11 Oct 2026 against a primary source (official page, repository file, dataset card, or the PS itself)
[S]	Comes only from the supplied research files; plausible but not re-verified
[D]	Our design decision (reasoned, not a fact)
[H]	Hypothesis to be tested by a named experiment
[U]	Unverified / open question; must be checked in the named phase before relying on it
[TARGET]	Pre-registered goal, not a result

1. Executive summary

The problem. On 26 August 2026 a glacier collapse in northern Nepal sent a flood of water, mud and debris down the Bhote Koshi–Trishuli corridor. Track B asks for a system that, given *any area and flood date chosen by the judges*, answers three questions from free satellite data: **where did the flood hit, what infrastructure was hit, and which settlements lost road access to the nearest town or hospital.** One AI component is required. We chose the **flood segmentation model** option, which has to be trained on the listed datasets and shown to work on Himalayan scenes it has never seen.

What we will build — "Cutoff". One pipeline runs from raw Sentinel-1 radar, with optional Sentinel-2, plus the Copernicus DEM and pre-event OpenStreetMap, to a map dashboard and a one-page situation report:

1. **Same-track pair selection.** For the AOI, find one post-event and up to three pre-event Sentinel-1 scenes from the *same relative orbit*. If none exist, say so instead of falling back silently.
2. **Hazard mapping, combining three evidence sources:**
   - a **U-Net (ResNet-34 encoder)** trained on **Kuro Siwo** with 7 inputs (pre-event mean VV/VH, post-event VV/VH, their differences, slope) that predicts *no water / permanent water / flood water*;
   - a **scene-adaptive change detector** for what the model was never taught (debris, scoured channels, smooth mud), whose threshold is estimated from terrain in the same scene that cannot have flooded;
   - **terrain gating** (radar shadow/layover, slope, HAND) and optional **Sentinel-2 corroboration** that *downgrade or upgrade* confidence. Neither one silently deletes a pixel.
3. **Exposure.** Hazard polygons are overlaid on pre-event OSM buildings, roads and bridges. Results are reported as **"exposed", never "destroyed"**.
4. **Access.** A properly noded road graph is solved under **three explicit closure scenarios**. Each settlement gets one of four statuses: *connected, degraded, cut off, unknown (no mapped route)*.
5. **Reporting.** `results.json` is the single source of truth. The dashboard and a template-generated situation report read only from it, so no number can be invented.

What changed after verification. Checking the earlier plans against primary sources changed several decisions (§7.3 and §9 give the full list):

- **[V]** Kuro Siwo totals **1.02 TB** (GeoTIFFs) or **1.96 TB** (webdataset), not the ~95 GB assumed earlier. We download a labelled-GRD subset by event, never the

whole dataset.
- **[V]** Kuro Siwo samples are **linear σ⁰** from a SNAP graph that uses a *Lee Sigma* filter, **SRTM** terrain correction and **no terrain flattening**. To keep training and inference consistent we must reproduce that radiometry. But SRTM is *not an allowed input* under the PS, so at inference we substitute the Copernicus DEM and record this as a known domain difference.
- **[V]** The official Kuro Siwo split is defined by **event (activation) IDs**: 27 train, 7 validation, 10 test. We reuse it, so every reported number is “held-out by event”.
- **[V]** Since June 2026, **Sentinel-1C and -1D** fly the same tracks 6 days apart. A same-track pair can therefore be 6 *or* 12 days apart; the PS tip mentions only 12.
- **[V]** GEOID-Flood and MMFlood are built from Copernicus EMS maps, so training on them risks disqualification. They are rejected, as are ImpactMesh and WorldFloods (EMS-derived or not on the PS list).
- **Sentinel-2 is not fused into the model [D].** No allowed dataset provides *labelled, temporally paired* pre/post Sentinel-1 + Sentinel-2. Sentinel-2 therefore enters as decision-level evidence, with explicit missing-modality handling (§10.3, §11.3–§11.4).

What we claim, and what we don’t. The contribution is **engineering and evaluation**, not a new architecture:

- terrain-aware hazard mapping with an honest held-out test that is *stratified by slope*;
- a zero-tuning transfer test on Trishuli against EMSR927 (check-only);
- a cut-off analysis that reports a *range* across closure assumptions and OSM gaps instead of a single false-certain number.

We do not claim state-of-the-art accuracy, operational readiness, validated damage classification, or uniqueness.

Feasibility. The MVP fits the 15-day window on free resources:

- a laptop with 16 GB RAM for the SAR/terrain/graph pipeline;
- roughly 4–8 hours of free Colab GPU for a 10–15k-patch Kuro Siwo subset (an estimate, to be measured).

The critical path is **data → S1 preprocessing → training → evaluation → integration → demo**. The single riskiest link is Sentinel-1 preprocessing latency during live judging, which is mitigated by caching, a lighter preprocessing path, and a question to the organisers (§22.3).

The deliverable you are reading also comes with a **tested starter code base** (13 synthetic-data unit tests passing). Every code listing in §18 comes from those files, so the code, tensor shapes and repository layout described here are consistent with each other.

2. Beginner-friendly overview

Imagine a valley after a disaster. The road is gone, the phone mast is down, and nobody knows which villages can still be reached. A satellite passes over every few days and takes a picture, even through clouds.

Two kinds of “camera”:

- **Radar (Sentinel-1)** sends microwaves down and listens for the echo. Calm water reflects the pulse *away*, like a mirror, so it looks **dark**. Rocky debris scatters the pulse in all directions, so it often looks **brighter**. Radar works at night and through monsoon cloud.
- **Optical (Sentinel-2)** is like a very good colour camera with extra infrared “colours”. Water looks distinctive in infrared, but optical **cannot see through cloud**, and the Himalayan monsoon is cloudy.

How Cutoff finds the flood. It compares a radar picture from *after* the flood with pictures from *before*, taken from exactly the same position in orbit. Where the valley floor suddenly got much darker or much brighter, something changed. A neural network trained on 40-odd floods around the world marks pixels that look like new flood water. A simple statistical rule marks unusual change the network was never taught about, such as debris. Elevation data removes impossible answers: water cannot sit on a 40° mountainside, and some slopes are hidden from the radar entirely (“radar shadow”).

From pixels to people. OpenStreetMap, as it was *before* the flood, tells us where the buildings, roads, bridges, villages and hospitals are. We overlay the flood map:

- buildings and roads inside it are **exposed**;
- road pieces under the flood are treated as **closed**;
- a route-finding algorithm checks whether each village can still reach a hospital or town.

Because we can’t be sure a flooded road is impassable, we run three versions of the analysis (cautious, moderate, bridges-only) and show the range.

What comes out. A map you can click, a table of villages with status *connected / degraded / cut off / unknown*, downloadable map files, and a one-page report in which every number comes from the map.

What it can’t do. It can’t warn before a disaster: satellites pass every few days, while a glacier collapses in minutes. It can’t see objects much smaller than 10 m. It can’t confirm a building is destroyed, only that it sits inside the mapped hazard. And

it is only as good as the pre-event map: a village whose path was never mapped gets “unknown”, not “cut off”.

3. Official problem statement and requirements

3.1 Source of authority

The two supplied PS documents (`Track_B.pdf` and the designed brief `Space_Track.pdf`) have **identical rules [V]**. They are the authority on eligibility. Everything else in this document is our interpretation or design.

3.2 The problem, users, and why it matters

- **Problem:** after a Himalayan glacier/debris flood, ground sensors, roads, bridges and communications fail. Responders need *where*, *what* and *who is cut off*, from space, quickly enough to matter.
- **Who experiences it:** search-and-rescue and logistics teams, district authorities, humanitarian coordinators, and the affected villages themselves. The PS frames the system as an **educational prototype, not an operational tool [V]**.
- **Operational challenges:**
  - persistent monsoon cloud, which blinds optical sensors;
  - steep terrain, which creates radar shadow and layover;
  - narrow channels relative to 10 m pixels;
  - satellite revisit gaps;
  - incomplete maps of remote valleys.

3.3 Requirements matrix (official vs ours)

ID	Requirement (official wording condensed)	M/O	Our implementation	Acceptance test
R1	Input: **an area and a flood date**; must run **for any area and date the judges choose**	M	CLI + dashboard accept a bbox/polygon and a date; nothing is hard-coded to Trishuli	Fresh run on two non-Trishuli AOIs (E7)
R2	**Where did the flood hit?** Map flooded **and debris-covered** areas **by comparing before/after images**; use **S1**, and **S2 when clear**	M	Same-track S1 change + U-Net + rule-based disturbance; S2 as optical evidence where clear	Hazard maps + EMSR927 comparison
R3	**What was damaged?** Overlay **OSM buildings, roads, bridges**, estimate which were hit	M	Hazard ∩ OSM by confidence tier; reported as “exposed”	Counts in `results.json`; EMS check

ID	Requirement (official wording condensed)	M/O	Our implementation	Acceptance test
R4	**Who is cut off?** Settlements with no road connection to the nearest **town or hospital**	M	Noded graph, 3 closure scenarios, 4 statuses	Synthetic-graph tests; scenario table (E6)
R5	**AI component: choose one** (segmentation model *or* sitrep copilot)	M	**Segmentation model**	—
R6	Train “using the listed training data” and **show how well it works on Himalayan scenes it has never seen**	M	Kuro Siwo only; zero-tuning Trishuli test + slope-stratified Kuro Siwo test	E1, E2, E4 tables
R7	Bonus: trace flood path downstream from any point using elevation; list settlements	O	pyflwdir D8 path + HAND corridor + OSM places	Path follows Bhote Koshi → Trishuli
R8	Case study: map the Aug 2026 Trishuli flood, **compare with EMSR927**	M	`eval/emsr927.py`, check-only	Agreement + disagreement taxonomy
R9	**Map dashboard + one-page situation report generated by the system**	M	Streamlit + Jinja2 template → PDF	Usability pass
R10	GitHub repo with README on how to run	M	`README.md`, `Makefile`, pinned env	Teammate fresh-clone run
R11	Report ≤ 6 pages incl. **limitations**	M	`docs/report/`	Page count
R12	3-minute demo video	M	Script in §19.5	Timed
R13	Required attributions + training-dataset citation	M	`ATTRIBUTION.md`, footers of dashboard, sitrep, report	Unit test on exact strings
R14	**Allowed inputs only:** S1, S2, Copernicus DEM, **pre-event OSM**, listed training datasets. **Not allowed as inputs:** CEMS, UNOSAT or any published damage map; post-event OSM edits. **Breaking this rule = disqualification**	M (critical)	Allow-list config; `src/` never imports `eval/` or reads `reference/`; OSM date guard	`test_no_reference_leak`, `test_osm_date_guard`
R15	OSM snapshot **before 26 Aug 2026** (latest available stated as 27 Jul 2026)	M	Default `2026-07-27`, asserted	Test

ID	Requirement (official wording condensed)	M/O	Our implementation	Acceptance test
R16	Modified OSM data, if published, stays **ODbL**	M	ODbL notice on every OSM-derived export	README + export metadata
R17	Tip: change detection only between **same orbit track** images	Strongly implied	Pair selector groups by relative orbit + direction	`test_pair_selector_rejects_cross_track`
R18	**Evaluation is live**, on judge-chosen areas/dates	M	Cached + fast path; timing tests	E7
R19	Respect: **no victim images**; prototype wording	M	Maps only; disclaimer on every output	Review

Scoring weights [V]: flood and damage mapping quality 30%; AI component 25%; cut-off settlement analysis 20%; report, Q&A understanding and honesty about limitations 15%; dashboard and sitrep usability 10%. Effort in §23 is allocated roughly in this proportion.

3.4 Inputs, outputs, constraints, scope

- **Input:** AOI (bbox or polygon in WGS84), event date, optional OSM snapshot date (default 2026-07-27), optional point for the bonus trace.
- **Output:**
  - hazard class raster (GeoTIFF, UTM) and polygons (GeoPackage);
  - per-pixel calibrated flood score and evidence flags;
  - exposure tables;
  - settlement status per scenario;
  - `results.json`, the dashboard, and a one-page PDF situation report.
- **Non-functional requirements:**
  - runs end-to-end without manual steps;
  - finishes on a judge-sized AOI within the live slot **[U: slot length unknown]**;
  - reproducible environment;
  - every number traceable to inputs and a model version;
  - honest uncertainty and limitations.
- **Explicit constraints:** the data rules (R14–R16), same-track comparison (R17), attribution (R13), and the 6-page and 3-minute limits.
- **In scope:** post-event mapping and access modelling, a Himalayan evaluation, and a demo.
- **Out of scope:**
  - early warning;
  - flood depth;
  - building-level damage grading;
  - real-time tasking of satellites;

- casualty estimates (we will never print casualty figures; the PS's NDRRMA numbers are context only).

Our additions (not required by the PS, each justified):

Addition	Why
Slope-stratified metrics	Strengthen R6
Calibration (temperature scaling)	Supports the honesty criterion
Closure scenarios and the "unknown" status	Defensible R4
OSM-dropout sensitivity	Shows how much R4 depends on map completeness
S2 corroboration layer	R2 literally asks for it "when the sky is clear"

4. Research methodology and source inventory

4.1 Method

1. Read every supplied file in full.
2. Extract each claim that a design decision depends on.
3. Check those claims against primary sources on 11 Oct 2026:
   - the official PS;
   - the Kuro Siwo repository files `README.md`, `LICENSE`, `configs/grd_preprocessing.xml`, `configs/train/data_config.json` and `requirements.txt`;
   - the Hugging Face dataset cards for both Kuro Siwo releases;
   - the CEMS EMSR927 news page;
   - the CDSE Sentinel-1 mission documentation and the CDSE news item on the 2026 orbital reconfiguration;
   - the Planetary Computer `sentinel-1-rtc` collection;
   - the pyflwdir API reference.
4. Label each claim [V]/[S]/[U].
5. Resolve contradictions (§7.3).

Limits of the search: we did not read full texts of most papers (abstract level, as the Elicit review itself warns); we could not query CDSE for actual 2026 Trishuli acquisitions; we did not inspect the EMSR927 vector packages. These remain [U] items with named phases.

4.2 Source inventory

Source	Type	Topics	Key findings used	Limitations / issues found	Influence on project
`Track_B.pdf`, `Space_Track.pdf`	Official PS	Requirements, data rules, datasets, licences, judging	All of §3	Images illustrative; live-round logistics not specified	Defines boundaries
`Cutoff_Track_B_project_blueprint.pdf` (10 Oct 2026, 32 pp.)	Prior AI-assisted design	Full architecture, compliance matrix, experiments E1–E8, repo, roadmap	Same-track pairing, terrain gating, scene-adaptive disturbance, closure scenarios, template sitrep, Kuro Siwo-only training	Assumed Kuro Siwo trainer flags and the 7-ch U-Net would just work; storage estimate unmeasured; Sentinel Hub fast path unconfirmed	Baseline design; most decisions retained

Source	Type	Topics	Key findings used	Limitations / issues found	Influence on project
`Cutoff_Hackathon_Proposal_Audit.pdf` (29 pp.)	Independent AI audit of the blueprint	Compliance, fact checks, issue register	Verdict “build after major corrections”: float16/subset storage, fast-mode preprocessing, OSM noding, softer HAND gating, WeasyPrint deps, attribution test	Some figures unsourced (e.g. “15–20 pp inflation from random splits”, “Colab ~78 GB disk”, specific dB thresholds); some EMSR927 contents (WorldView-3 dates, file names) not verifiable by us	Corrections adopted; unsourced figures labelled [U]
`Himalayan_Flood_Mapping_AI_Assessment` (16 pp.)	AI technical assessment	Physics, failure modes, conformal prediction, Himalayan folds	Physical failure modes (shadow/layover, wet mud ≈ water, mixed pixels, revisit latency); exposure ≠ damage; grounded copilot	Proposes training on Himalayan basins with **no labelled data identified**; Swin dual-stream + conformal in 15 days; thresholds cited to a water-quality paper; claims CEMS gives isolated-village labels **[U]**	Physics adopted; architecture and folds rejected (§7.3)
`Flood_Repo_Investigation_Report` (8 pp.)	AI repository survey	9 repos: Kuro Siwo, Sen1Floods11, ETCI-2021, ImpactMesh, Prithvi, UrbanSARFloods, Sentinel1-Flood-Finder, DAM-Net, OSMnx	Kuro Siwo has the best multitemporal SAR code; no repo does flood→OSM→isolation→report end-to-end; licence notes (GPL-3.0 reference only)	Recommends forking **ImpactMesh** (EMS labels ⇒ disqualification risk) and Prithvi (optical-only); “7/7” scores not reproducible; says Kuro Siwo has “NO DEM”, contradicted by repo **[V]**	Licence and gap notes used; ImpactMesh/Prithvi rejected
Elicit literature review (PDF) + sources sheet (16 rows)	AI-assisted literature synthesis	S1 segmentation, fusion, mountain debris, transfer, damage, accessibility, uncertainty, reporting	Dual-pol helps (+5 IoU pts vs rule chain), radiometric aug helps / geometric hurts (Helleis 2022); event-held-out testing essential; raster vs network access differ by ~300k people (Petricola 2022); exposure ≠ passability; 15-day scope advice	Abstract-level; PDF says 19 papers, sheet has 16	Experimental design and claims discipline

Source	Type	Topics	Key findings used	Limitations / issues found	Influence on project
Kuro Siwo repo `configs/grd_preprocessing.xml` **[V]**	Primary (code)	Preprocessing	Orbit → border noise → thermal noise → **Calibration σ⁰ linear** → **Lee Sigma** 7×7 → Terrain-Correction with **SRTM 1Sec**, **10 m**, **EPSG:3857**; no dB, no terrain flattening	—	Inference preprocessing must mirror this
Kuro Siwo repo `configs/train/data_config.json` **[V]**	Primary (code)	Splits, normalisation	train/val/test **activation IDs** (27/7/10); inputs pre1, pre2, post; channels VV, VH; `clamp_input 0.15`; mean [0.0953, 0.0264] (linear)	—	Our split; confirms linear units
HF dataset cards (GeoTIFFs, Webdataset) **[V]**	Primary	Files, size, licence	Per-sample `flood_vv/vh`, `sec1_vv/vh`, `sec2_vv/vh`, `dem`, `mask` (0/1/2), `valid_mask`, `info.json`; **1.02 TB / 1.96 TB**; **CC BY 4.0**	Units and `info.json` fields not documented	Dataset class; download plan
CEMS EMSR927 page **[V]**	Official	Case study	26 Aug 2026 flash flood “reportedly” GLOF, Rasuwa District; flood extent + damage assessment requested	AOIs, products and image dates not listed on that page	Check-only evaluation
CDSE S1 docs + orbital reconfiguration news **[V]**	Official	Data access, revisit	GRD and GRD-COG available; 1A ended 29 Jun 2026; **1C + 1D, 6-day revisit**	RTC section content not retrievable	Pair selector; latency planning
Planetary Computer `sentinel-1-rtc` **[V]**	Official	Fast-path option	γ⁰ RTC by Catalyst using **PlanetDEM**, CC BY 4.0, account required	DEM not Copernicus; processing differs from Kuro Siwo	Rejected as model input (§10.4)

5. Existing systems and literature review

5.1 Research papers and datasets

Work	Purpose / task	Imagery	Method	Reported results (original context)	Strengths	Weaknesses	What we reuse	Licence / attribution
Kuro Siwo — Bountos et al., NeurIPS 2024 D&B, arXiv 2311.12056 **[V]**	Multitemporal SAR flood segmentation / change detection	S1 GRD & SLC: 2 pre + 1 post; DEM	U-Net, SNUNet, FloodViT, change-detection baselines	Best models >80% F1 on flood, 10 held-out test events (abstract) **[S]**; DEM/slope gains marginal **[S]**	Expert photo-interpreted labels; event split; code + weights	Water only (no debris class); event geography lowland-heavy **[S]**; very large	Training data, split, preprocessing graph	Data CC BY 4.0; code MIT; cite paper
Sen1Floods11 — Bonafilia et al., CVPRW 2020 **[S]**	Single-date water segmentation	S1 + S2 chips, 512²	FCNN, thresholding	IoU figures in paper, per split	S1 and S2 co-registered; hand-labelled test	Single date ⇒ no change signal; weak labels from thresholds; no licence file (mirrors CC BY 4.0 per PS)	Optional ablation only (§11.6)	Cite; non-commercial educational use per PS
Helleis et al. 2022, IEEE JSTARS **[S]**	CNNs vs operational rule-based S1 processor	S1 amplitude	AlbuNet-34, FCN, DeepLabV3+, U-Net, U-Net++	Dual-pol +5 IoU pts vs rule chain; weighted CE/Lovász +2; radiometric aug helped, geometric hurt; two events IoU 0.96/0.94 (not comparable across papers)	Rigorous comparison	Lowland events	Augmentation policy; dual-pol	Cite
Bai et al. 2021, Remote Sensing **[S]**	S1+S2 fusion on Sen1Floods11	S1 + S2	DL with focal loss	Test IoU 52.3%, Bolivia IoU 76.7%	Shows fusion can help	Single-date; metric spread	Caution on cross-paper comparison	Cite
Yang et al. 2021 (Baige outburst flood), Remote Sensing **[S]**	Mountain outburst-flood mapping	S1 + S2 + DEM	Index thresholds when clear, SAR + terrain when cloudy, RF	OA 92.3%, κ 0.89; map within 5 days	Closest mountain analogue; per-pixel sensor switching	Single event	Idea of "optical where clear, SAR elsewhere"	Cite
GEOID-Flood (2026 preprint), **MMFlood** (IEEE Access 2022) **[V]**	Large multimodal benchmarks	S1, S2, DEM	Foundation-model benchmarking	Fusion best on transient water (abstract)	Scale, diversity	**Labels derived from CEMS maps** ⇒ forbidden as training data; GEOID spans 2016–2026, may include EMSR927	Related work only	—

Work	Purpose / task	Imagery	Method	Reported results (original context)	Strengths	Weaknesses	What we reuse	Licence / attribution
UrbanSARFloods (CVPRW 2024) **[S]**	Urban + open flood, SLC coherence	S1 SLC	Benchmarks	Urban flooding remains hard	Coherence	Not on PS list; no licence	Related work	—
Petricola et al. 2022 (Cyclone Idai) **[S]**	Healthcare-access loss	S1 flood masks + OSM	Raster vs network travel time	Methods differ by ~300k people (≈800k vs 500k); incomplete roads affect network estimates more	Directly analogous to R4	Flood as static binary mask	Scenario framing; “unknown” status	Cite
Warnier et al. 2020; **Kwant et al. 2021 [S]**	Network reachability / flood impact on roads	OSM + road status / S1	Graph analytics	Demonstrations	Operational framing	Not satellite-only	Reachability formulation	Cite
Kopiika et al. 2024 [S]	Tiered bridge damage	S1 coherence + VHR + crowdsourcing	Segmentation + rules	17-bridge case	Multi-scale design	Not satellite-only	Argument for “exposed ≠ damaged”	Cite
Guo et al. 2017 (ICML)	Calibration	—	Temperature scaling	—	Simple, argmax-preserving	Calibration may not transfer across domains	Calibrate on validation events	Cite

5.2 Repositories (code actually present vs claims)

Repo	What exists in code	What does not	Use in Cutoff	Licence
`Orion-AI-Lab/KuroSiwo` **[V]**	`main.py` trainer (U-Net, DeepLabV3, UPerNet, SNUNet, Siam-diff/conc, BIT, ChangeFormer, ConvLSTM, ViViT…), `dataset/Dataset.py`, SNAP graphs, split pickles, released FloodViT and SNUNet weights (Dropbox)	No OSM, routing, gating, dashboard	Data, split, preprocessing reference, B1 baseline weights	MIT (code), CC BY 4.0 (data)
`cloudtostreet/Sen1Floods11` **[S]**	Notebooks, GCS data	Change detection	Optional optical ablation	None in repo; PS: mirrors CC BY 4.0
`JakubNiedzwiedz/SAR-Flood-Detection` **[S]**	Refined Lee, change detection, JRC water removal, slope filter, OSM flooded roads	Learned model, benchmark eval	Design reference	Check before reuse
`AliForghani/sentinel1-sar-flood-mapping` **[S]**	ASF search, co-registration, change detection, Streamlit	Routing	Dashboard pattern	Check
`GIScience/healthcare-access-idai` **[S]**	S1 flood + ORS isochrones	Bridge logic, uncertainty	Method reference	Check

Repo	What exists in code	What does not	Use in Cutoff	Licence
`gboeing/osmnx` **[S]**	Graph building from OSM	Historical snapshots via ohsome	Optional helper; we build from ohsome GeoJSON	MIT
`cordmaur/Sentinel1-Flood-Finder` **[S]**	RF on Planetary Computer	DL	Reference only	**GPL-3.0** — do not copy code
`IBM/ImpactMesh`, Prithvi **[S]**	Multimodal training with EMS-derived labels; optical FM	—	**Rejected** (data rule / optical-only)	Apache-2.0

5.3 Operational systems

Copernicus EMS Rapid Mapping (analyst products from VHR imagery) is the reference for the case study. It is **not reproducible**, **not an allowed input**, and its definitions and dates may differ from ours. The Global Flood Monitoring service and UNOSAT products are likewise check-only. None of these publishes an automated *settlement isolation* layer we could compare against **[U]**.

5.4 What the landscape tells us

Each piece already exists separately: S1 change detection, U-Nets on Kuro Siwo, flood→road overlays, and healthcare-access modelling. What we did **not** find in one open workflow is the combination of:

- same-track pairing enforced automatically;
- terrain gating of a trained model;
- slope-stratified held-out evaluation;
- a zero-tuning Himalayan transfer test;
- closure-scenario isolation with “unknown” handling.

This is a statement about our search, not proof of absence.

6. Research gap and defensible novelty

6.1 Gap analysis

Gap	Evidence the gap exists	PS weight	Feasible in 15 days	Decision
G1 Mountain false alarms (shadow/layover/steep slopes) and no Himalayan held-out proof	PS R6; radar geometry physics; mountain studies are single cases (Elicit gap 3)	30% + 25%	Yes	**Core**
G2 Hazard → passability → isolation link with explicit assumptions	Petricola 2022 sensitivity; Elicit gap 4	20%	Yes	**Core**
G3 Debris vs water	Kuro Siwo has no debris class **[V]**; opposite backscatter signs **[S]**	Part of R2	Rule-based only (no labels)	Secondary, flagged [H]
G4 Calibrated uncertainty	Elicit gap 5	Honesty 15%	Light version	Included (temperature scaling, tiers)
G5 Grounded reports	Elicit: no evidence that generated prose improves outcomes	10%	Easy (template)	Deliverable, not a contribution
G6 S1+S2 learned fusion	No allowed paired labelled pre/post S1+S2 set	—	No (data)	Decision-level evidence instead
G7 Foundation-model fine-tuning	Unproven for debris; compute	—	No	Rejected

6.2 Comparison matrix

Capability	Existing approaches	Our approach	Evidence of improvement	Validation method
Change detection geometry	Often any available pre image	Same relative orbit enforced; cross-track only as labelled ablation	[H3]	E5: stable-steep FPR, same vs cross track
Mountain false positives	Fixed slope/HAND masks or none	Physical gating that *downgrades*; shadow/layover → “no observation”	[H2]	E2: FPR on slope > 15° and recall, by stratum
Non-water flood impacts	Water-only models	Scene-adaptive ±kσ disturbance in HAND corridor	[H6]	E4 per-class agreement with EMS observed event layers

Capability	Existing approaches	Our approach	Evidence of improvement	Validation method
SAR + optical	Fused models need paired data	Decision-level S2 evidence where SCL clear; explicit missing-modality states	[H7]	E9: tier changes vs EMS where S2 clear; optional Sen1Floods11 ablation
Generalisation	Random tile splits common	Official event split + slope strata + zero-tuning Trishuli	—	E1, E2, E4
Confidence	Raw softmax	Temperature scaling + evidence tiers + abstain class	[H8]	E8: ECE before/after
Isolation	Single binary answer	3 scenarios × 4 statuses + OSM dropout range	[H5]	E6
Usability	Masks in notebooks	Dashboard + sitrep from one JSON	—	Usability walkthrough

6.3 Minimum set of differentiators

The three we will defend:

1. **Same-track, terrain-gated hazard mapping with an honest held-out test stratified by slope.**
2. **A zero-tuning Himalayan transfer test with a disagreement taxonomy.** This explains *why* we differ from EMSR927 instead of only reporting a number.
3. **Range-based cut-off analysis**: three closure scenarios, the “unknown” status, and an OSM-dropout study.

For each: (1) existing approaches do the parts separately; (2) the remaining limitation is mountain false alarms and false certainty about isolation; (3) we implement these as tested modules; (4) this matters because these are the errors that would send rescuers to the wrong place; (5) we test them with experiments E2, E4, E5 and E6.

7. Final problem definition and objectives

7.1 Project specification

Item	Definition
Name	**Cutoff**
Tagline	*Where the valley changed, who lost the road, and how sure we are — from free satellite data.*
Problem	After a mountain flood, responders lack a fast, honest map of impact and access.
Solution	Same-track S1 change + trained U-Net + terrain gating (+ S2 when clear) → OSM exposure → scenario-based road-graph access → dashboard + sitrep.
Target users	Disaster-response analysts and coordinators (educational prototype)
Primary use case	Judge enters AOI + date → maps, exposure, cut-off table, sitrep within the live slot
Secondary uses	Post-event review; training material; bonus downstream trace from an upstream point
Main features	Pair selection; hazard classes with evidence flags; exposure; access statuses; dashboard; sitrep; exports
Differentiators	§6.3
Expected outputs	`hazard_class.tif`, `p_flood.tif`, `hazard.gpkg`, `exposure.gpkg`, `settlements_{A,B,C}.gpkg`, `results.json`, `sitrep.pdf`
Assumptions	Organisers allow cached/pre-downloaded scenes **[U]**; CDSE reachable; Kuro Siwo subset downloadable
Main risks	Live preprocessing latency; domain shift; OSM gaps; no debris labels (§23)
Success criteria	MVP checklist in §27.4 met; E1, E4, E6 tables produced; live run on a non-Trishuli AOI

7.2 Three explanations

- **For a beginner:** see §2.
- **For an ML engineer:** a 7-channel U-Net (ResNet-34 encoder, ImageNet-initialised, first conv adapted) is trained with CE + Dice on Kuro Siwo's official event split. Inputs are pre-event mean dB VV/VH, post dB VV/VH, dB differences and slope; labels are no-water / permanent / flood.
  - Logits are temperature-scaled on validation events, and tiled inference uses Hann-window blending.
  - A rule-based detector flags $|\Delta\sigma^0| > 3\sigma$, with σ estimated by MAD over HAND > 100 m terrain in the same scene, inside a HAND ≤ 60 m corridor.

    - Gating moves model detections on slope > 15° or HAND > 30 m to “possible” and masks shadow/layover as no-observation.
    - Sentinel-2 MNDWI under SCL-clear masks upgrades or downgrades tiers.
    - Evaluation is per event and per slope stratum, plus ECE, stable-steep FPR and EMSR927 agreement.
- **For judges (30 seconds):** “Rescuers need two answers: where did the valley change, and which villages lost their road. Cutoff gets both from free radar data for any valley and date, filters the false alarms mountains create, and shows a range of cut-off villages under stated assumptions instead of false certainty. We tested it on held-out floods and on Trishuli without tuning, and we’ll show you where it fails.”

7.3 Conflict-resolution table

#	Conflict	Evidence	Final decision	Rationale
C1	Repo report: fork **ImpactMesh** (S1+S2+DEM) vs PS data rules	ImpactMesh labels from CEMS **[S]**; PS forbids EMS products as inputs **[V]**	**Reject**	Disqualification risk outweighs benefit
C2	Train on **GEOID-Flood / MMFlood / WorldFloods** (Elicit, Doc B)	GEOID/MMFlood derived from CEMS **[V]**; WorldFloods not listed	**Reject**; related work only	Same rule; GEOID 2016–2026 may contain EMSR927
C3	Kuro Siwo licence: MIT vs CC BY	Repo LICENSE = MIT; README & HF cards: data **CC BY 4.0 [V]**	Code MIT, data CC BY 4.0; cite Bountos 2024	Both stated by owner
C4	Sen1Floods11 licence: “unlicensed, unclear” (repo report) vs “mirrors CC BY 4.0, fine for educational use” (PS)	PS is authority **[V]**	Allowed, optional, cite	PS rules
C5	Kuro Siwo size: ~94.8 GB float32 (audit) vs actual	HF: **1.02 TB** GeoTIFFs, **1.96 TB** webdataset **[V]**	Download labelled-GRD subset by event; budget 30–60 GB **[TARGET]**	Measured size dominates estimates
C6	Repo report: Kuro Siwo has “NO DEM”	`dem.npy` per sample; `--dem/--slope` flags **[V]**	DEM present; we derive slope ourselves	Primary source
C7	Radiometry: Doc C says use **γ^0 RTC** (terrain-flattened); Kuro Siwo uses **σ^0, no flattening**	`grd_preprocessing.xml` **[V]**	**Match Kuro Siwo (σ^0, Lee Sigma, 10 m)** at inference; terrain handled by gating + slope channel	Train/inference consistency beats physical nicety for a learned model
C8	Kuro Siwo terrain correction uses **SRTM**; PS allows only Copernicus DEM	**[V]** both	Use **Copernicus GLO-30** in our SNAP graph; record as known shift; test sensitivity (E3b)	Compliance first; difference is small vs speckle **[H]**

#	Conflict	Evidence	Final decision	Rationale
C9	Kuro Siwo output in **EPSG:3857**; area maths needs equal-scale CRS	**[V]**	Our pipeline outputs **UTM** (EPSG:32645 for Trishuli); areas refuse 3857 (tested)	Web Mercator inflates area by $1/\cos^2(\text{lat}) \approx 1.29$ at 28°N
C10	Revisit: PS tip “12 days apart” vs 6 days	1C + 1D same tracks, 6-day revisit since late Jun 2026 **[V]**	Pair by relative orbit + direction (platform may differ); 6 or 12 days	Both are same geometry
C11	HAND thresholds: 5 m / 15 m / 30 m (Doc C, audit) / 60 m corridor (blueprint)	No primary source for any value	Configurable: water-plausibility 30 m; disturbance corridor 60 m; stable terrain > 100 m; fixed **before** seeing EMSR927	GLO-30 smooths gorges (audit); E2 tests
C12	Fixed dB thresholds (−4/+3/+8 dB, Doc C)	Cited to a water-quality paper	**No fixed dB thresholds**; scene-adaptive kσ (k = 3)	Avoid unsourced constants
C13	Doc C: three-fold CV training on Himalayan basins	No labelled Himalayan flood training data identified	Kuro Siwo event split + Trishuli zero-tuning test	Can’t train on labels that don’t exist
C14	Doc C: Swin dual-stream + conformal + super-resolution	15 days; no debris labels; S2 unpaired	U-Net + temperature scaling + tiers; conformal = stretch	Feasibility
C15	Doc C: “isolated-village precision/recall vs CEMS maps”	No evidence CEMS publishes isolation layers **[U]**	Compare closed roads/bridges vs EMS damaged transport features if present	Only check that is possible
C16	Audit: “random patch splits inflate F1 by 15–20 pp”	Unsourced	Use event split; do not quote the number	Unsupported figure
C17	Audit: closure scenario A uses P > 0.3; blueprint uses P ≥ 0.5	Design choices	A = any hazard class 1–4 + 15 m buffer; B = ≥ 50 m in high-confidence; C = bridges only	Defined on tiers, not raw probabilities
C18	Audit: Sentinel Hub / RTC fast mode for live judging	Planetary Computer RTC uses PlanetDEM and different processing **[V]**; CDSE RTC details [U]	Fast mode = our own lighter graph on CDSE GRD-COG subset (§10.4); ask organisers before using any third-party RTC	Domain and rule risk
C19	Event cause and casualties differ (PS: glacier collapse; CEMS: “reportedly GLOF”)	**[V]** both	Never print casualty figures; use “reported”	Respect + accuracy
C20	Kuro Siwo events are indexed by **CEMS activation numbers** (e.g. 130, 470, 554)	`data_config.json` **[V]**	Allowed: the PS explicitly lists Kuro Siwo as training data, and its labels were produced by the Kuro Siwo team’s photo-interpretation (dataset card **[V]**). Note in the report	Transparency

7.4 MVP and optional features

MVP (must exist at submission):

1. Same-track pair selector.
2. S1 preprocessing (cached + live).
3. U-Net B3/P trained on a Kuro Siwo subset.
4. Disturbance layer.
5. Gating.
6. Exposure.
7. Graph with scenarios A and B and four statuses.
8. `results.json`, Streamlit dashboard and template sitrep.
9. `eval/emsr927.py`.
10. Experiments E1, E4, E6(i, iii).
11. Tests, README, report and video.

Strong build:

- E2, E3, E5, E7, E8;
- scenario C and the OSM dropout study;
- the S2 evidence layer;
- temperature scaling;
- the bonus flow path.

Stretch:

- a Nepali sitrep template reviewed by a native speaker;
- a conformal abstain class;
- the Sen1Floods11 optical ablation (E9b);
- a second Himalayan check scene;
- SLC coherence.

7.5 Research questions and hypotheses (all [H] until measured)

ID	Hypothesis	Experiment	Falsified if
H1	Pre-mean + post + difference beats post-only on held-out events	E3	Flood IoU gain < 2 pts
H2	Gating cuts false-positive area on slope > 15° by ≥ 30% with ≤ 5 pts recall loss	E2	Either bound missed
H3	Cross-track pairs raise stable-steep FPR vs same-track	E5	No difference beyond seed noise
H4	Flood F1 on Trishuli is within 15 pts of in-domain test	E1 vs E4	Larger drop (reported honestly)

ID	Hypothesis	Experiment	Falsified if
H5	The cut-off set changes materially across scenarios and 10–20% OSM dropout	E6	Set stable (then say so)
H6	Backscatter increases and decreases separate two hazard types on the valley floor	E4 per class	No class-wise agreement
H7	Where S2 is clear, S2 evidence improves precision of the “high” tier	E9	No precision change
H8	Temperature scaling reduces ECE on validation events	E8	ECE not reduced

8. Satellite technology fundamentals

8.1 Remote sensing in one paragraph

Remote sensing means measuring the Earth without touching it, usually by recording electromagnetic energy from a satellite. **Passive** sensors (Sentinel-2) record sunlight reflected from the surface. **Active** sensors (Sentinel-1) emit their own signal and record the echo. Each image pixel is a measurement for a patch of ground, 10 m × 10 m for the bands we use. The image comes with **georeferencing**: a coordinate reference system (CRS) plus an affine transform that maps pixel (row, col) to map coordinates. Lose that and the pixels no longer correspond to places.

8.2 Sentinel-1 and SAR

- **Sentinel-1** is the European C-band (≈ 5.4 GHz, ≈ 5.5 cm wavelength) radar mission. In 2026 the operating units are **Sentinel-1C and Sentinel-1D**. Sentinel-1A ended operations on 29 June 2026, and the C/D pair gives a **6-day** revisit on the same ground tracks **[V]**. Over land the standard mode is **Interferometric Wide swath (IW)**, about 250 km wide. We use the **Ground Range Detected (GRD)** product: amplitude only, about 10 m pixels after processing.
- **SAR (Synthetic Aperture Radar)** builds a sharp image by combining echoes recorded while the satellite moves, as if it had a very long antenna. Microwaves pass through cloud and need no sunlight, so **SAR works through monsoon cloud and at night**.
- **Backscatter (σ^0, "sigma-nought")** is the fraction of energy returned per unit ground area. It is usually shown in decibels: `dB = 10·log10(σ⁰)`. A linear value of 0.1 equals −10 dB. Typical values:
  - calm water: very low, often below −18 dB in VV, because the surface acts like a mirror and reflects energy away from the sensor (*specular reflection*);
  - rough ground or debris: higher (*diffuse scattering*);
  - buildings: often very high (*double bounce* between wall and ground).
- **Polarisation.** The radar transmits and receives in vertical (V) or horizontal (H) orientation. **VV** (vertical transmit, vertical receive) is most sensitive to surface roughness and water. **VH** (vertical transmit, horizontal receive, "cross-pol") responds to volume scattering such as vegetation and is noisier.
- **Speckle** is grainy interference noise in every SAR image. It is *multiplicative*, which is why we average and difference in dB, where it becomes additive.
- **Side-looking geometry** causes mountain artefacts:

    - **Foreshortening/layover:** slopes facing the radar are compressed, or folded onto other ground.
    - **Radar shadow:** slopes facing away and steeper than the look angle receive no signal. They look dark, *exactly like water*. This is the main false-alarm source in the Himalaya.
- **Relative orbit (track):** each pass follows one of the repeating ground tracks, identified by a number and a direction (ascending or descending). Two images from the **same** track see the terrain from the **same angle**, so a pixel-by-pixel difference measures change on the ground rather than change in viewing geometry. That is why the PS insists on same-track comparisons **[V]**.

8.3 Sentinel-2 and optical imagery

- **Sentinel-2** carries the MultiSpectral Instrument: 13 bands from visible to short-wave infrared (SWIR), at 10, 20 or 60 m. **Level-2A (L2A)** products are atmospherically corrected surface reflectance and include a **Scene Classification Layer (SCL)** that flags cloud, cloud shadow, snow, water and so on.
- **Useful bands for water:**
    - B03 green (10 m);
    - B08 NIR (10 m);
    - B11 SWIR-1 (20 m).

  Water reflects some green light but absorbs NIR and SWIR strongly, so:
    - **NDWI** = (G − NIR)/(G + NIR);
    - **MNDWI** = (G − SWIR1)/(G + SWIR1), which is positive over open water and suppresses built-up false positives better than NDWI.

  Sediment-laden flood water is brighter in red and SWIR, which lowers MNDWI. That is a known weakness to test.
- **Clouds, shadows, terrain, season.** Cloud hides the ground. Cloud shadow and terrain shadow are dark and can look like water. Snow is bright in visible light. Vegetation changes between seasons. During the monsoon, clear post-event Sentinel-2 views of the Trishuli corridor may be rare **[U]**, although EU Space published Sentinel-2 views of 12 and 27 August near Betrawati **[S]**.

8.4 Why SAR and optical are complementary — and why they can’t simply be stacked

What each sensor adds:

- SAR guarantees an observation through cloud and senses roughness and water.
- Optical adds spectral identity (water vs mud vs vegetation loss) but only when the sky is clear.

Why they can’t just be stacked:

1. They are acquired at **different times**, often days apart, while the flood recedes.
2. They come on **different grids and resolutions**: SAR is terrain-corrected to a DEM, and some Sentinel-2 bands are 20 m.
3. **Missing data differs per pixel**: cloud for optical, shadow/layover for SAR.
4. **Physics differs**: SAR brightness is not a colour.
5. Most importantly for us, a model learns a fused representation only from **paired labelled examples**. **No dataset allowed by the PS provides labelled, temporally paired pre/post S1+S2 [V for Kuro Siwo being SAR-only; S for Sen1Floods11 single-date]**.

Combining them correctly means co-registration, temporal-gap checks, cloud masks and a defined fallback for each missing modality. Cutoff does this at the **decision level**: S1 is mandatory, and S2 is optional evidence (§11.4).

8.5 Task vocabulary (not interchangeable)

Task	Question answered	Output	In Cutoff?
Flood detection	Is there a flood in this scene/tile?	Yes/no	No
Water-body classification	Which pixels are water (any water)?	Mask	Partially (permanent water class)
Flood segmentation	Which pixels are *flood* water at time t?	Per-pixel classes	**Yes (model)**
Flood change detection	Which pixels changed between pre and post?	Change mask	**Yes (model inputs + rule layer)**
Flood extent mapping	Georeferenced polygons of the flooded area	Vector + area	**Yes (output)**
Flood-depth estimation	How deep is the water?	Metres	**No** (out of scope)
Damage assessment	Is a structure damaged?	Grades	**No** — we report *exposure* only

9. Dataset comparison and selection

9.1 Candidate datasets

Property	Kuro Siwo (GRD)	Sen1Floods11	WorldFloods	Copernicus EMS (EMSR927)	GEOID-Flood / MMFlood
Official source	github.com/Orion-AI-Lab/KuroSiwo; HF `orion-ai-lab/Kuro-Siwo-GeoTIFFs`, `-Webdataset` **[V]**	github.com/cloudtostreet/Sen1Floods11 **[S]**	Mateo-Garcia et al. 2021 **[S]**	mapping.emergency.copernicus.eu **[V]**	arXiv 2608.02315; github.com/edornd/mmflood **[V]**
Publication	Bountos et al., NeurIPS 2024 D&B	Bonafilia et al., CVPRW 2020	Sci. Reports 2021 **[S]**	Analyst product	2026 preprint; IEEE Access 2022
Samples	43 events, 2015–2022; 67,490 labelled 224^2 patches **[S, consistent across audit + blueprint]**	4,831 chips 512^2, 11 events **[S]**	S2-based **[S]**	4 AOIs **[S]**	219 / 95 events
Sensors	**S1 only**: 2 pre + 1 post, VV+VH; DEM	S1 (single date) + S2	**S2 only**	VHR optical (+S1/S2)	S1 + S2 + DEM
Resolution	10 m	10 m	10 m	sub-metre source	10 m
Temporal	**Pre/post** (change-capable)	Single date	Single date	Post	Pre/post
Geography	6 continents, 3 climate zones; lowland-heavy **[S]**	6 continents	Global	Rasuwa, Nepal	Global
Labels	Expert photo-interpretation; 0 no-water, 1 permanent, 2 flood **[V]**	Hand + weak (thresholded)	Semi-automatic	Observed event, damage grading	**Derived from CEMS**
Paired S1+S2?	No	Yes, single date	No	n/a	Yes
Imbalance	Strong (flood minority)	Strong	—	—	—
Size	**1.02 TB GeoTIFFs; 1.96 TB webdataset [V]**	~14 GB **[S]**	large	small	large
Licence	Data **CC BY 4.0**, code MIT **[V]**	No file; mirrors CC BY 4.0 (PS)	—	Free, credit required	CC BY 4.0
PS status	**Listed (recommended)**	**Listed (optional)**	**Not listed**	**Check-only**	**Not listed; CEMS-derived ⇒ forbidden**

Property	Kuro Siwo (GRD)	Sen1Floods11	WorldFloods	Copernicus EMS (EMSR927)	GEOID-Flood / MMFlood
Fit for our model	**High**	Post-only baseline / optical ablation	None	Evaluation only	None

9.2 Final dataset strategy

Role	Dataset	Why	Exactly how used
Primary training + validation + in-domain test	**Kuro Siwo GRD (labelled)**	Only allowed dataset with *pre/post* S1 and expert labels; official event split **[V]**	Official `train_acts` (27) / `val_acts` (7) / `test_acts` (10)
Himalayan unseen test	**Trishuli 2026 scenes** that we process ourselves, compared with **EMSR927**	PS case study; never trained or tuned on	Zero-tuning E4
Optional baseline / ablation	**Sen1Floods11**	Only allowed S1+S2 paired data	B2-alt (post-only) and E9b (S1 vs S2 vs S1+S2, single-date)
Excluded	GEOID-Flood, MMFlood, ImpactMesh, WorldFloods, UrbanSARFloods	Rules / not listed	Related work only

Download first (Phase 2), in this order:

1. Kuro Siwo `catalogue.gpkg`, the event table with locations (Dropbox link in `download_kuro_siwo.sh` **[V]**). Use it to confirm event geography and that **no event lies within 100 km of 28.1°N 85.3°E**. Kuro Siwo covers 2015–2022, so the 2026 event cannot be in it.
2. A **listing** of the HF GeoTIFF repo files (`huggingface_hub.list_repo_files`), to learn the folder nesting **[U]**.
3. Labelled GRD samples for **3 validation events + 2 test events** (sanity), then the training subset.
4. Trishuli S1 scenes (same track; see §10), GLO-30 tiles, and OSM layers.

Training subset [D]. Stratify by event and flood fraction. Target **10–15k training patches**, ≈ 50% containing flood pixels, keeping every validation and test patch.

Storage per GRD patch (9 float32 arrays of 224^2 ≈ 196 kB each):

- uncompressed: 9 × 196 kB ≈ **1.8 MB**;
- 15k training patches ≈ **27 GB**;
- plus all validation/test patches **[U: count]**.

Converting to float16 `.npz` halves this. Budget **30–60 GB** of disk. These are our arithmetic, not measurements: measure in Phase 2.

Fallback if Kuro Siwo download is too slow:

- use the webdataset shards of only the needed events;
- else train on fewer events (report it);
- as a last resort, use the released **SNUNet** weights (B1) as the model and spend the effort on gating and evaluation. All PS questions are still answered.

9.3 Label harmonisation

We do **not** mix datasets in training. For evaluation against EMSR927:

- EMS “observed flood/event extent” polygons are rasterised to our 10 m UTM grid;
- our prediction “hazard” = classes {1 high flood, 3 rule-decrease, 4 rule-increase};
- permanent water and no-observation pixels are reported separately.

Damage-grading point layers, if present, are used only for the closed-road and closed-bridge check (E6-ii).

10. Data preprocessing

10.1 Why preprocessing must mirror Kuro Siwo

A learned model expects test inputs drawn from the same distribution as training. Kuro Siwo's verified graph is:

> Apply-Orbit-File → Subset → ThermalNoiseRemoval → Remove-GRD-Border-Noise → Land-Sea-Mask → **Calibration (σ^0, linear)** → **Speckle-Filter (Lee Sigma, 7×7 window, σ 0.9)** → **Terrain-Correction (SRTM 1Sec, 10 m, EPSG:3857, bilinear)** → GeoTIFF **[V]**

Our inference graph `graphs/s1_cutoff.xml` copies it with three deliberate changes:

1. **DEM = Copernicus 30 m** (SNAP's auto-download name for GLO-30 is to be confirmed in Phase 3 **[U]**), because SRTM is not an allowed input.
2. **Projection = local UTM**, for area-correct outputs. The model is fine with this because it sees 10 m pixels either way.
3. **No Land-Sea-Mask** (inland; it relies on SRTM).

We also export the **local incidence angle** and the **layover/shadow mask** from the terrain correction (SNAP Terrain-Correction can save these as bands **[U: exact option names]**).

10.2 Step-by-step pipeline

#	Step	Why	Tool	In → Out	Expected result / check
1	Acquisition	Get raw data	CDSE STAC + OData (`pystac-client`, `requests` with OAuth token); HF; ohsome; GLO-30	AOI, date → zips/COGs	Checksums; logged product IDs
2	Metadata inspection	Fail early	`rasterio_io.inspect`; STAC properties	file → dict	CRS present, expected bands, nodata known
3	Raster reading	Windowed reads keep RAM bounded	rasterio	GeoTIFF → ndarray + transform	Shapes match transform
4	Band selection	Only what the model saw	—	VV, VH (S1); B03, B08, B11, SCL (S2)	Missing VH ⇒ scene rejected (model needs both)

#	Step	Why	Tool	In → Out	Expected result / check
5	CRS verification	Wrong CRS = wrong place	pyproj	assert projected UTM for outputs	Area function refuses 3857/geographic (tested)
6	Spatial alignment	Pixel-wise comparison	`rasterio.warp.reproject` onto the **post-scene grid**	all layers → common 10 m grid	Same-track scenes align after TC; check NCC on stable terrain (target > 0.8 **[TARGET]**)
7	Temporal alignment	Physics changes with time	pair selector	scenes → pairs.json	Post within 12 d of event; pre within 48 d; S2 within ±5 d of S1 post or flagged
8	Resampling	Harmonise resolution	bilinear for continuous (DEM, S2 B11); nearest for categorical (SCL, masks)	→ 10 m	No categorical values invented
9	SAR normalisation	Match training	`to_db`, clip [−30, 5] dB; pre = mean of pre scenes in **linear** then dB; Δ = post − pre	→ 7-ch stack	Channel stats near training stats (log drift)
10	Optical cloud/shadow masking	Clouds look like anything	SCL ∈ {4,5,6,7,11} = clear; reflectance = (DN + offset)/10^4 (offset −1000 for baseline ≥ 04.00)	→ clear mask, MNDWI	Clear fraction reported
11	No-data handling	Never invent data	NaN propagation; `valid_mask`; shadow/layover → 254	→ class 254	“No observation” area reported
12	Ground-truth preparation	Training labels	Kuro Siwo `mask.npy`; invalid → 255 (ignore)	→ int64 labels	Label histogram per event
13	Patch extraction	Kuro Siwo is already 224^2; inference is tiled	`infer.predict_proba` (224, overlap 64)	AOI → tiles → mosaic	No seams (Hann blending)
14	Shape validation	Catch silent bugs	asserts in `build_input_stack` and the dataset	(7,224,224) / (224,224)	Tested
15	Split	No leakage	official event IDs; exclude events within 100 km of Trishuli	→ index CSV per split	No event in two splits (assert)

#	Step	Why	Tool	In → Out	Expected result / check
16	Augmentation	Robustness	radiometric gain ±1 dB, Gaussian noise σ 0.05 (normalised units); flips only in ablation	train only	Val/test never augmented
17	Class imbalance	Flood is rare	flood-biased sampling (~50% patches with flood), class weights (1, 1, 3), Dice	→ sampler	Per-class recall tracked
18	Quality checks	Trust	plot 20 random patches per split; stats per channel; % NaN	→ `outputs/qa/`	Human sign-off in Phase 4

10.3 Handling missing or unsuitable Sentinel-2

Situation	Detection	Behaviour
No S2 scene within ±5 days of S1 post	Catalogue search empty	S2 layer “not available”; tiers unchanged; flagged in `results.json`
Scene exists but cloudy	SCL clear fraction over AOI < 10% (configurable)	Only clear pixels used; others unchanged
Temporal mismatch > 5 days	dates	Shown on the map as context only; not used to change tiers
Snow / terrain shadow	SCL 11 / low illumination	Snow counts as clear but MNDWI is unreliable on snow → no upgrade on snow pixels

The model never receives S2. **Missing S2 can never break a run.**

10.4 Live-judging preprocessing modes [D]

Mode	When	What	Typical time
Cached	Trishuli and rehearsed AOIs	Preprocessed rasters stored in `cache/` by product ID	seconds
Full	New AOI, enough time	Full SNAP graph on an AOI **subset** (Subset node first, so only the AOI is processed)	**[U]** measure in E7; audit estimate 8–15 min per full scene
Fast	New AOI, short slot	Same operators but GRD-COG windowed read + Python calibration with the annotation LUT + Lee filter + GLO-30 geocoding (e.g. `sarsen` **[U]**)	**[U]**

The Fast mode must pass a **consistency check** before use: on Trishuli, its dB values should match Full mode within 0.5 dB median over stable terrain **[TARGET]**. Any third-party RTC product (e.g. Planetary Computer γ^0 with PlanetDEM) is **not** used without organiser approval (C18).

11. Model comparison and final selection

11.1 Candidates

Candidate	What it does / input → output	Pros	Cons	Data needs	Compute	Difficulty	Hackathon fit	Evidence
B0 Threshold change detection (Otsu or robust on Δ dB; permanent water and HAND masks)	$\Delta\sigma^0$ → binary	No training; transparent; works anywhere	Fixed thresholds fail on shadows, wet soil, vegetation	None	CPU, seconds	Low	**Baseline (required)**	Long operational history **[S]**
Random Forest on per-pixel features (dB, Δ, slope)	Pixel features → class	Fast, CPU	No spatial context; speckle-sensitive	Kuro Siwo	CPU	Low	Optional	Sentinel1-Flood-Finder, Baige study **[S]**
U-Net (ResNet-18/34 encoder)	Multi-channel image → per-pixel classes	Standard, reproducible; good at small structures; supported by `smp`	Needs GPU for training	Kuro Siwo	T4 OK	Medium	**Primary**	Helleis 2022: architecture differences small; data representation matters more **[S]**
DeepLabV3+	Same, atrous context	Good boundaries	Heavier; no clear advantage at 10 m	Same	Higher	Medium	Ablation only	**[S]**
Siamese / change-detection nets (SNUNet, Siam-diff, BIT)	Pre and post streams → change	Explicit temporal modelling; Kuro Siwo weights released for SNUNet	Input format must match released weights; more code	Kuro Siwo	Medium	Medium–High	**B1 baseline (released weights)**	Kuro Siwo benchmarks **[S]**
Dual-branch S1/S2 fusion	S1 and S2 encoders fused	Exploits optical	**No allowed paired pre/post labelled S1+S2**; S2 cloudy in monsoon	Paired labels (unavailable)	High	High	**Rejected** for core; E9b ablation on Sen1Floods11 single-date only	Bai 2021 single-date **[S]**
Foundation models (Prithvi, TerraMind)	Pretrained ViT + head	Data-efficient	Optical-centric or trained with EMS labels; heavy	—	≥ 16 GB VRAM **[S]**	High	Rejected	—

11.2 Final decision [D]

Primary model: U-Net with ResNet-34 encoder, single-date multi-temporal *input* segmentation, trained on Kuro Siwo with 7 channels:

Ch	Content	Units	Why
0, 1	Pre-event mean VV, VH	dB	Baseline state; mean of 2 pre scenes in linear units (lower speckle)
2, 3	Post-event VV, VH	dB	Current state
4, 5	Post − pre, VV and VH	dB	Explicit change cue; removes static terrain brightness
6	Slope	degrees	Terrain context (scale-invariant; raw DEM excluded because absolute elevation shifts massively between lowland training and Himalaya)

Task definition. One forward pass per location produces **3-class segmentation** (no water / permanent water / flood water) from a **pre/post input stack**. So it is *multi-temporal-input segmentation*, which is a form of change detection. We do not mix this with a separate pure change-detection network. The rule-based disturbance layer is a separate, labelled product.

Tensor contract:

- **Input:** `float32 [B, 7, 224, 224]`. Each channel is z-normalised with the mean/std computed on **training events only** (stored in the checkpoint).
- **Output:** logits `float32 [B, 3, 224, 224]` → `softmax(z / T)`, where `T` is the temperature fitted on validation events.
- **Decision:** a pixel is “flood (model)” if p(flood) ≥ τ. τ starts at 0.5 and is tuned on **validation events** only to maximise flood F1. τ and T are frozen before any Trishuli or EMSR927 data is touched.

Why this choice:

- **Dataset compatibility:** it uses exactly the modalities Kuro Siwo provides.
- **The PS:** it is a trained segmentation model that can be tested on unseen Himalayan scenes.
- **Feasibility:** `smp.Unet` handles the 7-channel first conv (tested).
- **Hardware:** it trains on a T4.
- **Evidence:** published comparisons find small architecture differences.

Rejected:

- the dual-branch S1/S2 model (no paired labels);
- transformers and foundation models (time, compute, rule risk);
- DeepLabV3+ as primary (no expected gain at 10 m; kept as an ablation).

Unverified assumptions:

- ImageNet initialisation helps 7-channel SAR **[H, ablation E3]**;
- SRTM → GLO-30 at inference is a negligible shift **[H, E3b]**.

11.3 Missing modalities

Missing	Behaviour
VH absent (single-pol scene)	Scene rejected (`NoValidPair` reason "needs VV+VH"); no zero-filling
Only 1 pre scene	Allowed: pre-mean = that scene (Kuro Siwo has 2; record `n_pre` and flag "reduced pre averaging")
No same-track pre scene	Run aborted with "no valid observation" + next expected pass; **never** a cross-track substitute
Shadow/layover / NaN pixels	Probabilities set to NaN → class 254 "no valid observation"
No S2	See §10.3; model unaffected

11.4 Hazard product (model + rules + gating)

Final classes (written to `hazard_class.tif`, uint8):

Code	Class	Rule
1	Flood water — high	Model p(flood) ≥ τ, slope ≤ 15°, HAND ≤ 30 m (or upgraded by S2 agreement)
2	Flood water — possible	Model flood but steep or high HAND, or downgraded by clear-sky S2 showing no water
3	Water / smooth mud (rule)	Δ < −kσ inside HAND ≤ 60 m corridor, not model flood
4	Roughness / debris (rule)	Δ > +kσ inside corridor
5	Permanent water	Model p(permanent) ≥ τ
254	No valid observation	Shadow/layover/NaN

The evidence bitfield (`flags.tif`) records *why*. The dashboard shows the reason on click.

What the confidence means. `p_flood` is a *temperature-scaled model score*. It is calibrated on Kuro Siwo validation events, **not** on the Himalaya, so it is not a probability of flooding on the ground in Nepal. The dashboard therefore shows **tiers** (high / possible / rule-only / no observation) and only shows `p_flood` as a secondary layer, labelled "model score, calibrated on global validation floods".

11.5 Baselines

ID	Baseline	Purpose
B0	Robust change threshold + permanent-water removal + HAND mask	Non-learned reference
B1	Released Kuro Siwo SNUNet weights, zero-shot (after matching its input format **[U]**)	Published reference
B2	U-Net post-only (ch 2–3)	Tests H1
B3	U-Net pre + post + diff (ch 0–5)	Core
P	B3 + slope + gating (+ S2 evidence)	Proposed

11.6 Optional optical ablation (E9b, stretch)

On **Sen1Floods11** hand-labelled data, train three small U-Nets on the same split: S1 only (VV, VH), S2 only (B03, B08, B11), and S1+S2 early fusion. This answers “does optical help when it is available” with real paired data. It does **not** transfer to the pre/post model and is reported as a separate finding.

12. Training methodology

12.1 Starting configuration (to be tuned; not results)

Parameter	Start value	Why it matters / justification
Patch size	224 × 224	Native Kuro Siwo patch; no resampling
Batch size	16 (T4 fp16); 4 on CPU smoke tests	Memory; BatchNorm stability needs ≥ 8
Epochs	≤ 30 with early stopping (patience 6)	Subset size; Colab session limits
Optimiser	AdamW	Robust default for fine-tuning pretrained encoders
Learning rate	3e-4, cosine decay	Common for ResNet-34 U-Nets; tune in {1e-4, 3e-4, 1e-3}
Weight decay	1e-4	Mild regularisation
Loss	CE (class weights 1, 1, 3) + soft Dice on flood	Imbalance; Dice optimises overlap directly
Augmentation	Radiometric gain ±1 dB + Gaussian noise; **no flips by default**	Helleis 2022: radiometric helped, geometric hurt **[S]**; flips change apparent look direction
Sampling	~50% patches containing flood	Imbalance
Early stopping	Validation flood IoU (event-level)	Directly the target metric
Checkpointing	`last.pt` every epoch, `best.pt` on improvement; resumable	Colab disconnects
Mixed precision	`torch.autocast` + GradScaler on CUDA	~2× speed, half memory
Seeds	42 (+ 7, 1234 if time)	Report mean ± std over seeds
Validation frequency	Every epoch	Cheap at this size
Calibration	Temperature scaling on validation logits (subsampled)	ECE reporting

12.2 Staged plan

1. **Baseline:** B0 on validation and test events (no training).
2. **Overfit test:** train on 32 patches until near-zero loss. This proves the data → model path works.
3. **Initial training:** B3 on the subset, one seed.
4. **Validation and debugging:** curves, a confusion matrix, and a 20-patch error gallery (shadows? permanent water? label noise?).
5. **Ablations (E3):** post-only, +pre, +diff, +slope, flips on/off, ResNet-18 vs 34, ImageNet vs scratch.
6. **Tuning:** LR × class weight (small grid), on **validation events only**.
7. **Final selection:** best validation flood IoU, then fix τ and T.

8. **Independent test:** run once on the Kuro Siwo test events, then once on Trishuli. **No re-tuning afterwards.**

12.3 CPU-only vs GPU plan

Stage	CPU-only (16 GB laptop)	Free GPU (Colab T4, if available)
Data prep, B0, gating, graph, dashboard	Yes	—
Overfit test (32 patches)	Yes (minutes)	—
Full subset training	Impractical (days) **[U]**	~4–8 GPU-h total incl. ablations **[U, estimate]**
Inference on 20 × 20 km AOI	Yes (≈ $2000^2/224^2 \approx 100$ tiles; minutes **[U]**)	Faster

Free Colab GPU time is **not guaranteed**. Mitigations:

- keep the data on Google Drive in float16 shards;
- resume from `last.pt`;
- fall back to ResNet-18 and fewer epochs;
- if no GPU is available at all, ship **B1 + gating** (B1 needs no training) and report that.

13. Evaluation strategy

13.1 Metrics in plain language

Take a 10 × 10 tile with 20 truly flooded pixels. The model marks 25 pixels as flood, 15 of them correctly.

Metric	Formula	Example	Meaning for responders
Precision	TP/(TP+FP)	15/25 = 0.60	Of what we call flooded, how much is — false alarms waste trips
Recall	TP/(TP+FN)	15/20 = 0.75	Of what is flooded, how much we found — misses are dangerous
F1 / Dice	2TP/(2TP+FP+FN)	30/45 = 0.67	Balance of both
IoU	TP/(TP+FP+FN)	15/30 = 0.50	Overlap ÷ union; harsher than F1
mIoU	mean IoU over classes	—	Overall, but dominated by easy “no water”
Confusion matrix	counts truth × prediction	—	Shows *which* confusion (e.g. permanent ↔ flood)
Stable-steep FPR	flood predictions on slope > 20° and HAND > 100 m ÷ such pixels	—	Mountain false-alarm rate **without needing labels**
Boundary F1 @ 30 m	F1 of boundaries with 3-pixel tolerance	—	Fair to 10 m SAR vs sub-metre reference
ECE	Σ bins	acc − conf	× weight
Inference time, peak RAM	wall-clock per AOI size	—	Live judging feasibility

Primary metric: flood-class IoU, per event. Recall is also reported with priority, because a missed flooded village is worse than a false alarm in rescue triage. The cut-off analysis turns that asymmetry into scenario A (conservative).

13.2 Protocol

- **Splits:** the official Kuro Siwo event split; Trishuli is never seen. Per-event results are reported; pooled numbers are always shown next to the per-event spread.
- **Strata:** slope bins 0–5°, 5–15°, > 15° (slope from DEM), so terrain failures can’t be averaged away.
- **Freezing:** τ, T, k, the HAND thresholds and the closure rules are fixed and committed (git tag `frozen-v1`) **before** `eval/emsr927.py` is first run.
- **EMSR927 comparison** (E4):
  - rasterise the EMS observed-event polygons onto our grid;

- compute pixel P/R/F1/IoU and boundary F1 over valid pixels;
- report the fraction of the reference that fell in “no observation”;
- classify disagreements by cause: cloud/timing, radar geometry, DEM/gorge, definition mismatch (EMS optical debris vs SAR water), OSM.
- Disagreement is **not automatically our error**: EMS is analyst-made from VHR optical, possibly on a different date.

13.3 Experiments

ID	Question	Data	Compared	Output
E1	Held-out quality	Kuro Siwo test events	B0, B1, B2, B3, P	Per-event + pooled table
E2	Does terrain gating help?	E1 by slope bin	B3 vs P	Stratified table + stable-steep FPR
E3	Which inputs/choices help?	Validation (+ test once)	post-only, +pre, +diff, +slope, flips, encoder, init	Ablation table (seeds)
E3b	SRTM vs GLO-30 terrain correction shift	One Kuro Siwo event re-processed both ways **[U: needs raw scenes]**	P	Δ IoU
E4	Himalayan transfer, zero tuning	Trishuli vs EMSR927	P vs B0 vs B1	Pixel, boundary, per AOI + taxonomy
E5	Same-track importance	One Trishuli AOI	same vs cross-track pre	Stable-steep FPR
E6	Cut-off robustness	Trishuli graph	(i) synthetic unit graphs; (ii) closed-edge vs EMS damaged transport features; (iii) scenarios A/B/C; (iv) 20 runs × 10% and 20% edge dropout	Status-change table, flip rate
E7	Live feasibility	10, 20, 40 km AOIs	pipeline	time + RAM
E8	Calibration	Validation events	T = 1 vs fitted T	ECE overall and per slope bin
E9	S2 evidence value	Trishuli AOIs with clear S2	tiers with vs without S2	Precision of “high” vs EMS
E9b	Optical ablation (stretch)	Sen1Floods11	S1 / S2 / S1+S2	IoU

13.4 Results templates (fill after experiments — no numbers yet)

E1 / E2 (per event × slope bin):

Model	Event	Slope bin	Precision	Recall	F1	IoU	Support px
B0	…	0–5°					
P	…	> 15°					

Model	Event	Slope bin	Precision	Recall	F1	IoU	Support px
P	ALL	all	mean ± sd over seeds				

E4 (Trishuli vs EMSR927):

AOI	Model	Valid px	P	R	F1	IoU	Boundary F1 @30 m	Ref in no-obs (%)	Main disagreement causes

E6 (access):

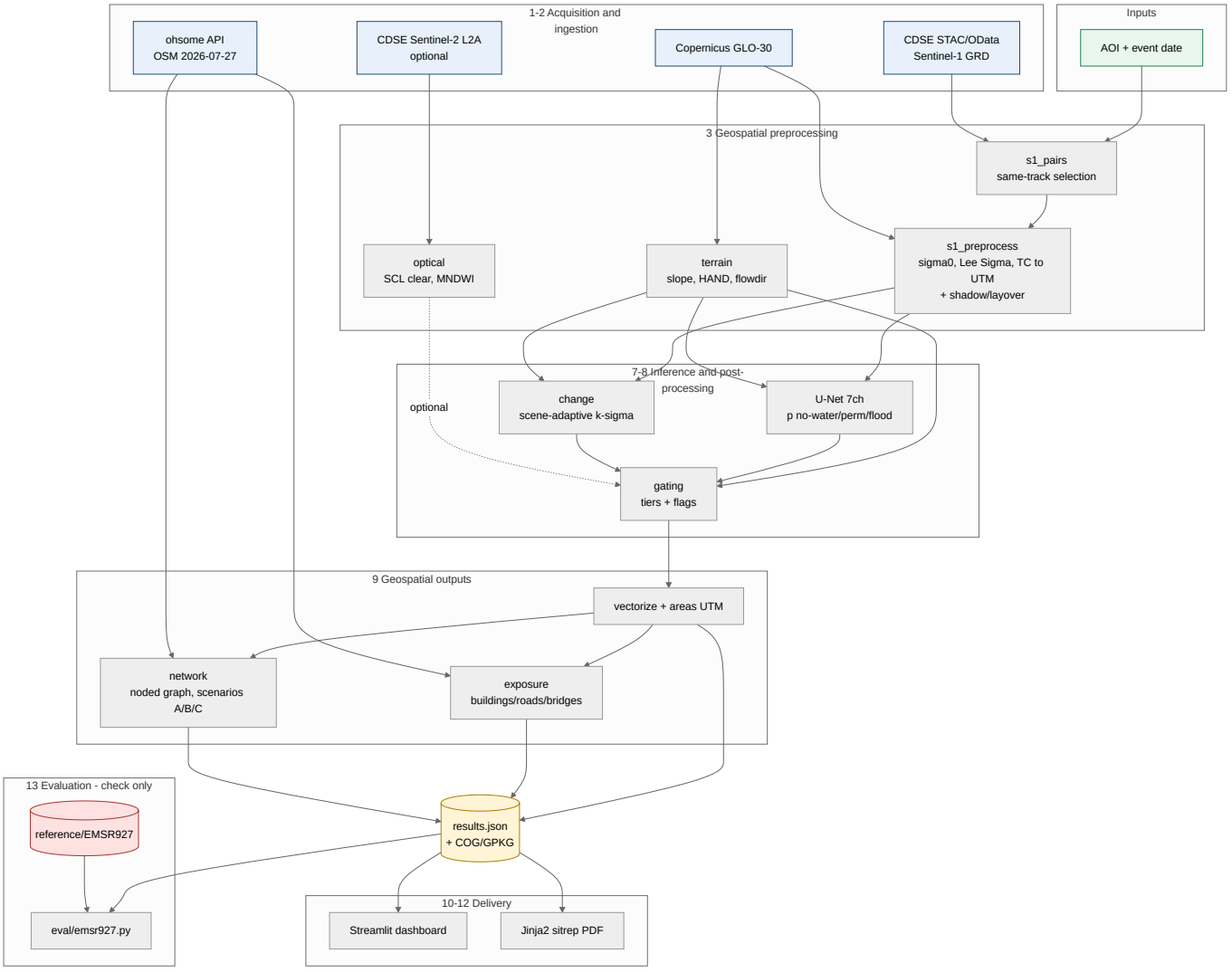
Scenario	Closed road km	Connected	Degraded	Cut off	Unknown	Jaccard vs A	Flip rate under 10% / 20% dropout

13.5 Interpretation rules

- Better pooled IoU but worse on > 15° slopes → claim the improvement only for flatter terrain.
- Good Kuro Siwo score but poor Trishuli score → report it as domain shift, show the failure classes, and do **not** retune.
- Exposure counts are “exposed”; damage is never inferred from a 10 m SAR cue alone.
- A cut-off settlement is always reported **with its scenario**.

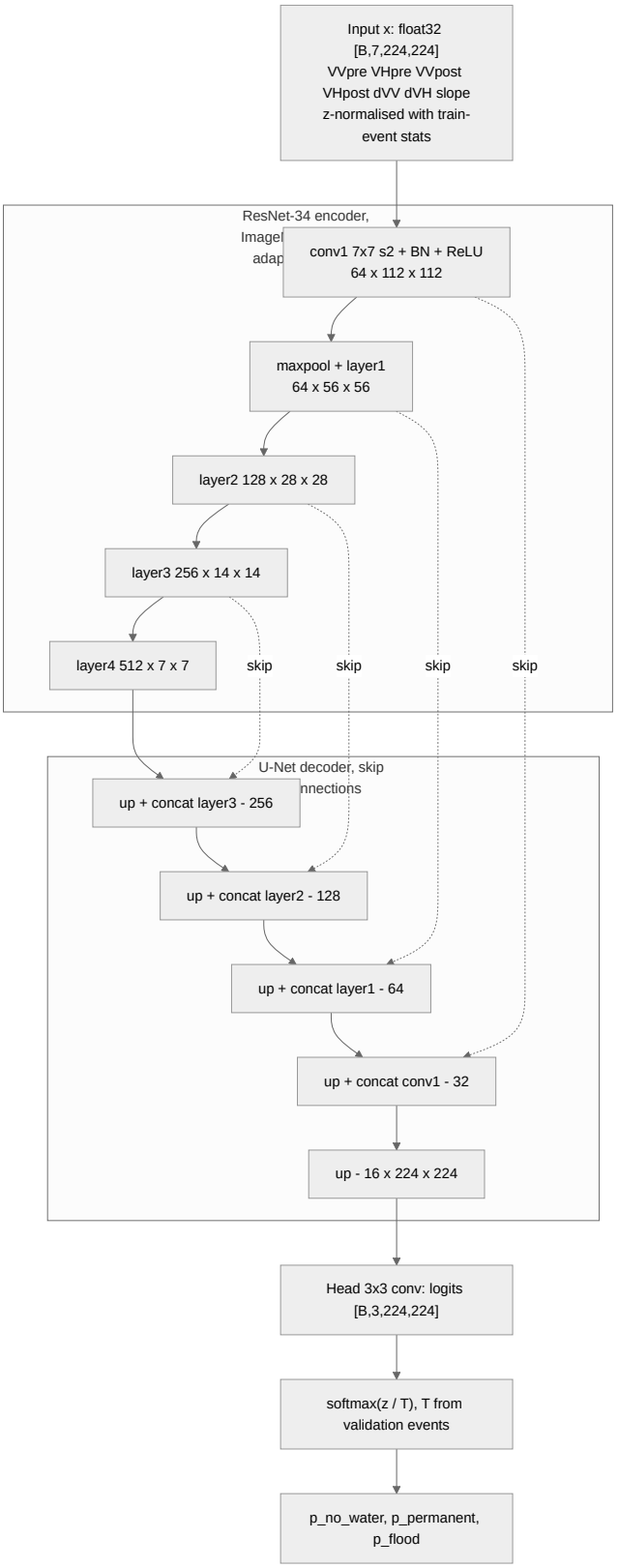
14. System architecture

14.1 System architecture

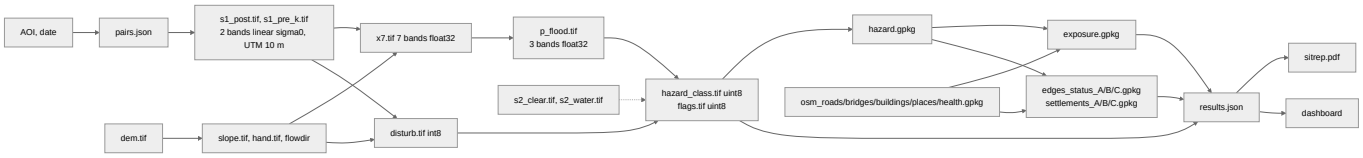


Model artefacts (`models/cutoff_unet_v1.pt` with normalisation stats, T and τ) are produced offline in training (§12) and loaded read-only by U-Net inference. `eval/` is the only reader of `reference/`, and nothing flows from evaluation back into the pipeline.

14.2 Model-level architecture



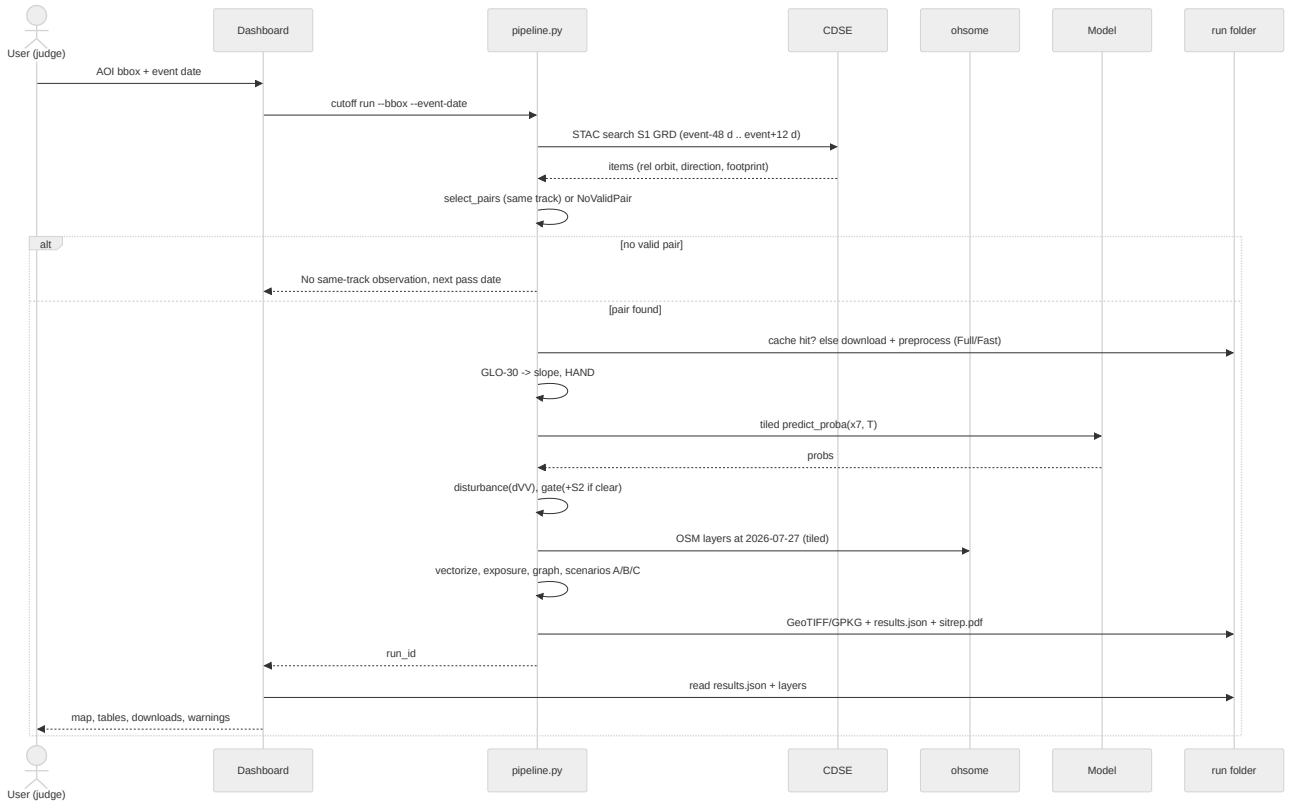
14.3 Data flow and artefacts



Contracts between stages:

Stage → stage	Contract (checked by `pipeline.py`; a violation stops the run with a message)
pair → preprocess	`pairs.json`: `rel_orbit`, `direction`, `post.id`, `pre[].id`, datetimes, coverage ≥ 0.8; post ≥ event date
preprocess → model	Rasters share CRS (UTM), transform and shape; bands VV, VH; dtype float32; nodata NaN
terrain → model/gating	`slope.tif` and `hand.tif` on the same grid; HAND in metres; slope in degrees
model → gating	`p_flood.tif` 3 bands summing to 1 where valid, NaN where invalid
gating → vector	uint8 codes {0–5, 254} only
vector/OSM → network	All layers in the same projected CRS; OSM date < event date
everything → results	`Results` pydantic model validates; any missing field fails the run

14.4 Inference sequence



14.5 Walk-through of a real user scenario

1. **Input.** The judge draws a 20 × 20 km box around Syapru Besi and enters 26 Aug 2026. The dashboard calls `cutoff run`.
2. **Find observations (needs new imagery).**
   - STAC search over 48 days before to 12 days after the event.
   - Scenes are grouped by `(relative orbit, direction)`. A track qualifies only with ≥ 1 post scene and ≥ 1 pre scene covering ≥ 80% of the AOI with VV+VH.
   - The best track is the earliest post scene, then coverage. The choice and alternatives are logged.
   - If none qualify, the run stops with a clear message. **Real-time availability is never implied:** the post scene may be up to 6–12 days after the event.
3. **Preprocess.** On a cache hit (e.g. Trishuli) the rasters are loaded. Otherwise the AOI subset is run through the Full or Fast graph, producing σ^0 (linear) VV/VH on a 10 m UTM grid plus a shadow/layover mask. GLO-30 tiles are mosaicked and resampled bilinearly to the same grid, and slope and HAND are computed.
4. **Model.** The 7-channel stack is built with the *same* `build_input_stack` used in training and normalised with the checkpoint's stats. Tiled inference (224, overlap 64, Hann blending) produces calibrated probabilities. Invalid pixels become NaN.
5. **Rules and gating.**
   - σ_stable is estimated from ΔVV over HAND > 100 m terrain, and pixels with |z| > 3 inside HAND ≤ 60 m are flagged.
   - Gating assigns tiers and flags.
   - If a clear S2 scene exists within ±5 days, MNDWI evidence adjusts the tiers.
6. **Vectorise and measure.** Classes are converted to polygons (specks under 5 px removed, simplified at 5 m). **Area is computed in UTM metres**: pixel count × |a·e| of the affine transform. This is valid because UTM is conformal with a scale error of ~0.04–0.1% within a zone. The function refuses Web Mercator and geographic CRSs (tested).
7. **Exposure.** Buildings whose footprint intersects a hazard class, road length inside each class, and bridges within 15 m are counted.
8. **Access.**
   - Roads are noded at crossings and densified to ≤ 100 m pieces; bridges are kept grade-separated.
   - The graph is solved with multi-source Dijkstra from hospitals and towns before and after closures, for each scenario.
   - Each settlement gets a status: *unknown* if it had no route even before the flood, *cut off*, *degraded* (> 1.5× travel time), or *connected*.
9. **Write.**
   - GeoTIFFs keep CRS and transform (rasterio profile) and carry tags with the model version, product IDs and the attribution string.
   - GeoPackages carry the CRS.

    - GeoJSON exports for web use are reprojected to EPSG:4326 per RFC 7946; OSM-derived ones carry the ODbL notice.
    - `results.json` is validated by `Results`, and the sitrep is rendered from it.
10. **Show.** The dashboard lists the inputs with dates and orbit, the map layers, tables, warnings (“S2 not available”, “12% of AOI no radar observation”, “3 settlements unknown: no mapped route”) and downloads.

Existing vs new imagery. Steps 3–10 run on any already-downloaded scenes. Only step 2 and the download part of step 3 require network access and new imagery.

14.6 Error handling and fallbacks

Failure	Detection	Fallback / message
CDSE down / quota	HTTP errors, retries with backoff	Use cache; else “satellite archive unreachable”
No same-track pair	`NoValidPair`	Stop with tracks seen + next pass; cross-track only via an explicit `--allow-cross-track` flag in ablation mode, watermarked “not comparable”
SNAP OOM / crash	exit code, missing output	Retry with smaller subset → Fast mode → cached
CRS mismatch	`inspect` + stage contract	Reproject at stage boundary; fail if no CRS
Invalid checkpoint	`load_checkpoint` raises	Stop; “model file invalid”
Empty prediction	zero hazard px	Valid outcome; report “no change detected” + no-observation area
ohsome timeout	HTTP 5xx	Tile smaller; cached OSM; exposure/access skipped with warning
No hospital/town snaps to graph	`access_status` raises	Widen snap radius once; else report “targets unreachable in OSM”
Dashboard can’t render big raster	size check	Serve downsampled PNG overlay; COG for download

15. Technology stack

Tool	What it is	Why / where	Alternative considered	Cost / access	Phase
Python 3.11 + conda-forge env	Language + packages	Everything	—	Free	All
PyTorch + **segmentation-models-pytorch**	DL framework + U-Net builder	Model, training, inference	Kuro Siwo trainer (heavier, wandb-bound, torch 1.13)	Free	Train, infer
rasterio (GDAL inside), **pyproj**	Raster I/O, reprojection	All raster steps	GDAL CLI	Free	All
geopandas, **shapely 2**	Vector ops	Vectorise, exposure, noding	PostGIS (overkill)	Free	Infer
numpy, **scipy**	Arrays, morphology	Everywhere	xarray (not needed at this scale)	Free	All
ESA SNAP (GPT) 10+	SAR processor	Match Kuro Siwo radiometry	`sarsen/pyroSAR` (Fast mode), ASF HyP3 (external)	Free; needs ~8–16 GB RAM	Preprocess
pystac-client, **requests**	Catalogue search + download	CDSE	`sentinelsat` (retired API)	Free account	Acquire
pyflwdir	D8 flow, HAND, path	Terrain, bonus	richdem, pysheds, WhiteboxTools	Free	Preprocess, bonus
NetworkX	Graph algorithms	Access analysis	igraph (faster), OSMnx (no historical snapshot)	Free	Infer
ohsome API	Historical OSM	Pre-event OSM (PS-named)	Overpass attic	Free, rate-limited	Acquire
Streamlit + **folium/streamlit-folium**	Python web app + Leaflet map	Dashboard	FastAPI + React + MapLibre (more work, no gain for judging)	Free	Deploy
Jinja2 + **WeasyPrint** (or headless Chromium)	HTML template → PDF	Sitrep	ReportLab	Free (WeasyPrint needs pango/cairo)	Report
pydantic	Schema validation	`results.json`	dataclasses	Free	All
pytest, GitHub Actions	Tests, CI	Leak test, unit tests	—	Free	All
Google Colab (free tier)	GPU notebook	Training	Kaggle notebooks (also free GPU quota **[U]**), local GPU	Free, quota not guaranteed	Train
Git + GitHub	Version control	Repo deliverable	—	Free	All

A. Essential MVP: Python, PyTorch, smp, rasterio, pyproj, geopandas, shapely, numpy/scipy, SNAP, pystac-client, pyflwdir, NetworkX, ohsome, Streamlit + folium, Jinja2 + PDF renderer, pydantic, pytest, Colab, GitHub.

B. Optional: `sarsen` (Fast mode), Docker (reproducible demo machine), OSMnx (convenience), torchmetrics, wandb (experiment tracking; a CSV/JSON log is enough).

C. Excluded:

- **Google Earth Engine:** extra account and terms; hides preprocessing; harder to show “runs from raw data”.
- **PostGIS / any database:** files are enough at AOI scale.
- **Microservices / FastAPI backend:** Streamlit calls the pipeline directly.
- **Paid APIs / LLMs:** the template sitrep needs none.
- **Foundation models:** rule risk and compute.
- **Planetary Computer RTC:** different DEM and processing; not confirmed as an allowed input.
- **TorchGeo:** useful, but it adds an abstraction we don’t need for one dataset with a custom layout.

16. Repository structure

```
cutoff/
├── README.md                    [MVP] how to install, download, run, reproduce
├── ATTRIBUTION.md               [MVP] three PS strings + Kuro Siwo citation + EMS credit (eval only)
├── LICENSE                      [MVP] e.g. MIT for code; data licences listed separately
├── environment.yml              [MVP] pinned conda-forge env (+ requirements-lock.txt)
├── Makefile                     [MVP] make setup | test | data-trishuli | train | run | eval | app
├── pyproject.toml               [MVP]
├── configs/
│   ├── default.yaml             [MVP] thresholds (tau, k, HAND, slope), paths, OSM date
│   ├── kuro_siwo_splits.yaml    [MVP] official train/val/test activation IDs + excluded events
│   ├── train_unet.yaml          [MVP] hyper-parameters
│   └── closure_scenarios.yaml   [MVP] A/B/C definitions
├── graphs/s1_cutoff.xml         [MVP] SNAP graph adapted from Kuro Siwo (cited)
├── src/cutoff/
│   ├── cli.py                   [MVP] `cutoff run|train|eval-kuro` entry points
│   ├── pipeline.py              [MVP] stage orchestration + contracts + caching
│   ├── data/ cdse.py s1_pairs.py s1_preprocess.py dem.py osm.py s2.py kuro_siwo.py
│   ├── geo/  sar.py terrain.py change.py gating.py optical.py rasterio_io.py
│   │         exposure.py network.py flowpath.py[bonus]
│   ├── ml/   model.py losses.py metrics.py train.py calibrate.py infer.py
│   └── report/ schema.py sitrep.py
├── eval/                        (never imported by src/)
│   ├── kuro_eval.py             [MVP] E1/E2/E3 tables
│   ├── emsr927.py               [MVP] E4 check-only
│   └── access_eval.py           [Strong] E6 dropout + closure replay
├── app/streamlit_app.py         [MVP]
├── notebooks/colab_train.ipynb  [MVP] thin wrapper around cutoff.ml.train
├── tests/                       [MVP] test_core.py (13 tests) + integration tests
├── scripts/                     fetch_kuro_subset.py, fetch_dem.py, run_trishuli.sh
├── docs/                        report/, limitations.md, data_rules_audit.md, experiment_log.md
├── reference/                   (git-ignored) EMSR927 files — read ONLY by eval/
├── cache/ data/ outputs/ models/  (git-ignored)
```

16.1 File responsibilities

File	Responsibility	Inputs → outputs	Key functions	Depends on	Test
`data/s1_pairs.py`	Same-track pair selection	STAC items → ranked pairs	`from_stac`, `select_pairs`, `NoValidPair`	—	cross-track rejection (done)
`data/cdse.py`	Search + authenticated download + cache	AOI, dates → files	`search_s1`, `search_s2`, `download(product_id)`	pystac-client	mocked HTTP
`data/s1_preprocess.py`	Run SNAP graph (Full) or Fast path on AOI subset	GRD → σ^0 VV/VH GeoTIFF + shadow mask	`preprocess(product, aoi, mode)`	graphs/	Trishuli stable-terrain NCC
`data/dem.py`	GLO-30 tiles → mosaic on grid	AOI → `dem.tif`	`fetch_glo30`, `to_grid`	rasterio	bounds/CRS
`data/osm.py`	ohsome snapshot by layer, tiled, date-guarded	bbox, date → GeoJSON	`fetch_layer`, `assert_pre_event`	requests	date guard (done)
`data/s2.py`	L2A search, SCL clear, MNDWI on grid	AOI, date → bool rasters	`best_clear_scene`, `evidence`	optical.py	synthetic SCL

File	Responsibility	Inputs → outputs	Key functions	Depends on	Test
`data/kuro_siwo.py`	Index, split filter, dataset, stats	sample folders → tensors	`index_samples`, `KuroSiwoDataset`, `compute_channel_stats`	sar.py, terrain.py	shapes (done)
`geo/sar.py`	dB, pre-mean, 7-ch stack (**shared train/infer**)	arrays → (7,H,W)	`to_db`, `build_input_stack`	—	done
`geo/terrain.py`	slope, HAND, flowdir	DEM → rasters	`slope_degrees`, `hand_and_flowdir`	pyflwdir	synthetic V-valley
`geo/change.py`	Scene-adaptive disturbance	ΔdB, HAND → int8	`disturbance`	—	done
`geo/gating.py`	Tiers + flags	probs, dist, terrain, S2 → uint8	`gate`	—	done
`geo/rasterio_io.py`	Inspect, write, areas, vectorise	—	`inspect`, `write_geotiff`, `class_areas_km2`, `vectorize`	rasterio	done
`geo/exposure.py`	Hazard ∩ OSM	polygons + OSM → counts	`exposure(hazard, buildings, roads, bridges)`	geopandas	synthetic
`geo/network.py`	Noding, graph, scenarios, statuses, dropout	roads, places, targets, hazard → statuses	`node_lines`, `build_graph`, `closed_segments`, `access_status`, `edge_dropout`	NetworkX	done
`geo/flowpath.py`	Bonus downstream trace	point, DEM → line + settlements	`trace(point)`	pyflwdir	synthetic DEM
`ml/model.py`	Build/save/load model with meta	—	`build_model`, `save_checkpoint`, `load_checkpoint`	smp	done
`ml/losses.py`, `ml/metrics.py`	CE+Dice; confusion metrics, strata, FPR, ECE	—	`CEDiceLoss`, `ConfusionMeter`, `StratifiedMeter`	—	done
`ml/train.py`, `ml/calibrate.py`	Train loop, temperature	datasets → `best.pt`, T	`train`, `evaluate`, `fit_temperature`	above	overfit test
`ml/infer.py`	Tiled inference	(7,H,W) → (3,H,W)	`predict_proba`	model	done
`report/schema.py`, `report/sitrep.py`	Results schema; template sitrep	dict → HTML/PDF	`Results`, `render_html`	pydantic, jinja2	render test
`pipeline.py`, `cli.py`	Orchestrate + contracts + cache keys	AOI, date → run folder	`run(aoi, date, mode)`	all	end-to-end on cached Trishuli
`eval/emsr927.py`	Check-only comparison	run + reference → JSON	`rasterize_reference`, `boundary_f1`	metrics	synthetic polygons
`app/streamlit_app.py`	UI	run folder → web page	—	streamlit	manual + smoke

Entry points and run order:

1. `make setup`
2. `make test`
3. `scripts/fetch_kuro_subset.py`
4. `cutoff train --config configs/train_unet.yaml` (Colab)
5. `cutoff eval-kuro` (E1–E3, E8)
6. `cutoff run --bbox ... --event-date ...` (pipeline)
7. `python eval/emsr927.py --run outputs/<id> --reference reference/...` (E4, after freezing)
8. `streamlit run app/streamlit_app.py`

17. Detailed implementation plan

Day numbers assume a 15-day window and 3–4 people (roles: **ML**, **SAR/terrain**, **OSM/graph**, **App/report**). If you are fewer, follow the critical path and drop the items marked Strong/Stretch.

Phase (days, owner)	Objective	Prerequisites	Tasks & files	Commands	Deliverable / acceptance	Common errors → fix
1 Environment & repo (D1, all)	Reproducible base	GitHub, conda	`environment.yml`, `pyproject.toml`, `Makefile`, `ATTRIBUTION.md`, CI with `pytest`	`conda env create -f environment.yml; make test`	Fresh clone passes 13 tests on two machines	GDAL wheels → use conda-forge for rasterio/gdal; torch CPU vs CUDA mismatch → separate Colab install cell
2 Data acquisition & verification (D1–3, SAR + ML)	All inputs reachable and understood	CDSE account; HF access	`data/cdse.py`, `scripts/fetch_kuro_subset.py`; download `catalogue.gpkg`, list HF files, fetch 5 events; Trishuli STAC listing; GLO-30; ohsome test	`python scripts/fetch_kuro_subset.py --events val,test --dry-run`	**Data inventory** (`docs/data_inventory.md`): Kuro Siwo layout + `info.json` keys + units (expect linear σ^0, mean ≈ 0.095 VV); Trishuli same-track candidates with relative orbits; DEM tiles; OSM counts	Huge downloads → `allow_patterns` per event; CDSE token expiry (refresh); ohsome 504 → smaller tiles
3 Geospatial preprocessing (D2–5, SAR)	Trishuli pre/post rasters on a UTM 10 m grid + terrain	Phase 2	`graphs/s1_cutoff.xml` (copy Kuro Siwo, swap DEM, UTM, export shadow/layover), `s1_preprocess.py`, `dem.py`, `terrain.py`	`gpt graphs/s1_cutoff.xml -Pinput=... -Poutput=...`	Pre/post align: NCC on stable terrain > 0.8 **[TARGET]**; HAND looks right along the Trishuli (visual)	SNAP OOM → Subset first, `-Xmx` in `gpt.vmoptions`; DEM name wrong → check SNAP DEM list
4 Data loader & sanity (D3–4, ML)	Trustworthy tensors	Phase 2 data	`kuro_siwo.py`, `configs/kuro_siwo_splits.yaml`, stats script, QA plots	`python -m cutoff.data.kuro_siwo --qa`	20 patches/split plotted; per-channel stats saved; **no event in two splits** (assert)	Event key wrong → read real `info.json`; NaNs → valid_mask

Phase (days, owner)	Objective	Prerequisites	Tasks & files	Commands	Deliverable / acceptance	Common errors → fix
5 Baseline (D4–5, ML)	B0 numbers + B1 attempt	Phase 4	`eval/baselines.py` (B0 threshold), B1 loader	`cutoff eval-kuro --model B0`	E1 rows for B0 (and B1 if format resolved)	B1 input format unknown → read `dataset/Dataset.py` in the repo
6 Primary model (D5, ML)	Model builds, overfits	Phase 4	`model.py`, `losses.py`	`pytest -k model`; overfit 32 patches	Loss → ~0 on 32 patches	Wrong channel count → assert `len(CHANNELS)==7`
7 Training & validation (D5–8, ML)	B2, B3, P trained	GPU	`train.py`, `calibrate.py`, `notebooks/colab_train.ipynb`	`cutoff train --config configs/train_unet.yaml --seed 42`	`best.pt` + history; validation flood IoU > B0 **[TARGET]**	Colab disconnect → resume `last.pt`; NaN loss → check dB clip, LR
8 Evaluation & ablations (D8–11, ML)	E1, E2, E3, E8 tables	Phase 7	`eval/kuro_eval.py`, `ablations.py`	`cutoff eval-kuro --strata slope`	Tables + 10-image failure gallery	Pooled hides strata → always print strata
9 Inference pipeline (D6–9, SAR + ML)	Probabilities + rule layer + gating on Trishuli	Phases 3, 7	`infer.py`, `change.py`, `gating.py`, `optical.py`, `s2.py`	`cutoff run --bbox <trishuli> --event-date 2026-08-26 --stages hazard`	`hazard_class.tif` + flags; stable-steep FPR logged	Seams → overlap/Hann; shadow mask missing → export from TC
10 Geospatial outputs (D8–10, OSM)	Areas, polygons, exposure	Phase 9 (or mock hazard from D6)	`rasterio_io.py`, `exposure.py`	`--stages vector,exposure`	GPKGs in UTM + 4326 GeoJSON with ODbL notice	Area in 3857 → refused by test
11 Access analysis (D6–11, OSM)	Statuses under A/B/C; dropout	OSM layers; mock hazard first	`network.py`, `access_eval.py`	`--stages access`	`settlements_{A,B,C}.gpkg`; E6 table	Whole-way closures → noding + densify (tested); villages off-graph → "unknown"
12 Backend/report (D9–11, App)	`results.json` + sitrep	Phases 9–11 or mocks	`schema.py`, `sitrep.py`, `pipeline.py`, `cli.py`	`cutoff run ...`	Sitrep PDF from mock JSON in < 5 s	WeasyPrint libs missing → `apt install libpango-1.0-0 libpangoft2-1.0-0` or Chromium print
13 Frontend & map (D9–12, App)	Dashboard	Mock run folder from D6	`app/streamlit_app.py`	`streamlit run app/streamlit_app.py`	All §19 panels working on the mock, then the real run	Heavy rasters → PNG overlays
14 Integration (D12–13, all)	One command, two AOIs	All	contracts, caching	`make run AOI=trishuli`; `make run AOI=other`	Two complete run folders; E7 timing table	Contract drift → pydantic + stage asserts

Phase (days, owner)	Objective	Prerequisites	Tasks & files	Commands	Deliverable / acceptance	Common errors → fix
15 Testing & debugging (D13–14, all)	Robustness	Phase 14	§20 test matrix; leak test in CI	`make test`	All tests green; failure drills done	Hidden local paths → grep for `/home/`
16 Freeze → EMSR927 → deploy/demo → docs (D13–15, all)	Honest case study and submission	Phase 15	tag `frozen-v1`; `eval/emsr927.py`; video; report ≤ 6 pp	`git tag frozen-v1`; `python eval/emsr927.py ...`	E4 table + taxonomy; video ≤ 3:00; report; README fresh-clone check	Temptation to retune after E4 → forbidden; record in log

Critical path (short hackathon): 2 → 3 → 7 → 9 → 14 → 16. Phases 10–13 run in parallel on **mock data from day 6**, so integration isn't left to the end.

If the team is short of time, in priority order:

1. B1 or B0 instead of a trained model (the rest still works).
2. Scenario A only.
3. No S2.
4. No bonus.

Continued development (post-hackathon):

- conformal abstain class;
- SLC coherence for urban and debris areas;
- more Himalayan check events (2025 Rasuwa outburst flood, Chamoli 2021) if reference polygons exist;
- reviewed Nepali templates;
- a Docker image;
- an evaluation harness at settlement level.

18. Code-level design and core examples

All listings below are the actual starter files. They run together and are covered by `tests/test_core.py`: **13 tests passed** on synthetic data, Python 3.11, torch 2.x CPU. Paths that depend on data we have not downloaded yet, such as the `info.json` event key and STAC field names, are isolated in clearly marked adapters.

18.1 SAR radiometry shared by training and inference — `geo/sar.py`

File: `src/cutoff/geo/sar.py`

```
        """SAR radiometry helpers shared by training (Kuro Siwo) and inference (CDSE scenes).

        Kuro Siwo GRD arrays are linear sigma0 (verified: configs/grd_preprocessing.xml
        calibrates to Sigma0 with no dB conversion; data_config.json mean VV=0.0953).
        Every path in Cutoff converts linear -> dB with the SAME function so that training
        and inference preprocessing cannot drift apart.
        """
        from __future__ import annotations

        import numpy as np

        DB_FLOOR = -30.0  # dB; values below this are clipped (near the S1 noise floor)
        DB_CEIL = 5.0


        def to_db(linear: np.ndarray, floor: float = DB_FLOOR, ceil: float = DB_CEIL) -> np.ndarray:
            """Linear sigma0 -> dB, clipped. NaN / non-positive values become `floor`."""
            lin = np.asarray(linear, dtype=np.float32)
            with np.errstate(divide="ignore", invalid="ignore"):
                db = 10.0 * np.log10(lin)
            db = np.where(np.isfinite(db), db, floor)
            return np.clip(db, floor, ceil).astype(np.float32)


        def pre_mean_db(pre_stack_linear: np.ndarray) -> np.ndarray:
            """Mean of several pre-event acquisitions, averaged in LINEAR space then converted.

            pre_stack_linear: (T, C, H, W) with T>=1. Averaging in linear units is the
            physically correct multi-look; averaging dB values biases low.
            """
            if pre_stack_linear.ndim != 4 or pre_stack_linear.shape[0] < 1:
                raise ValueError(f"expected (T,C,H,W) with T>=1, got {pre_stack_linear.shape}")
            return to_db(np.nanmean(pre_stack_linear, axis=0))


        def build_input_stack(pre_stack_linear: np.ndarray, post_linear: np.ndarray, slope_deg: np.ndarray) ->
np.ndarray:
            """Build the 7-channel model input (float32, C x H x W).

            Channel order (fixed, recorded in configs/channels.yaml):
              0 VV_pre_db  1 VH_pre_db  2 VV_post_db  3 VH_post_db
              4 dVV_db     5 dVH_db     6 slope_deg
            """
            pre = pre_mean_db(pre_stack_linear)  # (2,H,W)
            post = to_db(post_linear)  # (2,H,W)
            if pre.shape != post.shape or pre.shape[0] != 2:
                raise ValueError(f"pre {pre.shape} and post {post.shape} must both be (2,H,W)")
            if slope_deg.shape != pre.shape[1:]:
                raise ValueError(f"slope {slope_deg.shape} must match spatial shape {pre.shape[1:]}")
            diff = post - pre
            x = np.concatenate([pre, post, diff, slope_deg[None].astype(np.float32)], axis=0)
            return x.astype(np.float32)


        CHANNELS = ["vv_pre_db", "vh_pre_db", "vv_post_db", "vh_post_db", "dvv_db", "dvh_db", "slope_deg"]
```

18.2 Terrain — `geo/terrain.py`

File: `src/cutoff/geo/terrain.py`

```
        """Terrain derivatives from Copernicus GLO-30 (resampled to the 10 m analysis grid)."""
        from __future__ import annotations

        import numpy as np


        def slope_degrees(dem: np.ndarray, pixel_size_m: float) -> np.ndarray:
            """Slope in degrees via central differences. Edges use one-sided differences."""
            z = np.asarray(dem, dtype=np.float64)
            if z.ndim != 2:
                raise ValueError("dem must be 2-D")
            z = np.where(np.isfinite(z), z, np.nanmean(z) if np.isfinite(z).any() else 0.0)
            gy, gx = np.gradient(z, pixel_size_m)
            return np.degrees(np.arctan(np.hypot(gx, gy))).astype(np.float32)


        def hand_and_flowdir(dem: np.ndarray, transform, nodata: float = -9999.0, stream_upa_km2: float = 1.0):
            """Height Above Nearest Drainage with pyflwdir (D8).

            Returns (hand_m, flw). Streams = cells with upstream area > stream_upa_km2.
            Signatures checked against pyflwdir docs (from_dem, upstream_area, hand);
            verify on the pinned version in Phase 3.
            """
            import pyflwdir

            flw = pyflwdir.from_dem(data=dem.astype(np.float32), nodata=nodata, transform=transform, latlon=False)
            upa = flw.upstream_area(unit="km2")
            hand = flw.hand(drain=upa > stream_upa_km2, elevtn=dem.astype(np.float32))
            return hand.astype(np.float32), flw
```

18.3 Dataset class — `data/kuro_siwo.py`

File: `src/cutoff/data/kuro_siwo.py`

```
        """Kuro Siwo GRD dataset wrapper.

        Verified file layout per sample folder (HF dataset card, orion-ai-lab/Kuro-Siwo-GeoTIFFs):
          flood_vv.npy flood_vh.npy sec1_vv.npy sec1_vh.npy sec2_vv.npy sec2_vh.npy
          dem.npy mask.npy valid_mask.npy info.json
        mask: 0 no water, 1 permanent water, 2 flood.  valid_mask: 1 valid.
        Official event split (configs/train/data_config.json in the Kuro Siwo repo):
          train_acts / val_acts / test_acts  -> see configs/kuro_siwo_splits.yaml

        UNVERIFIED until Phase 2: the exact directory nesting and the info.json fields
        (event id key, coordinates). `index_samples` therefore takes the event id from a
        caller-supplied function; inspect one sample and set `event_key` in the config.
        """
        from __future__ import annotations

        import json
        from dataclasses import dataclass
        from pathlib import Path
        from typing import Callable, Sequence

        import numpy as np
        import torch
        from torch.utils.data import Dataset

        from cutoff.geo.sar import build_input_stack
        from cutoff.geo.terrain import slope_degrees

        IGNORE_INDEX = 255
        REQUIRED = ["flood_vv", "flood_vh", "sec1_vv", "sec1_vh", "sec2_vv", "sec2_vh", "dem", "mask",
"valid_mask"]


        @dataclass(frozen=True)
        class Sample:
```

```
            path: Path
            event_id: str


        def index_samples(root: Path, event_of: Callable[[Path], str]) -> list[Sample]:
            """Find sample folders (those containing mask.npy) and tag each with its event id."""
            out = []
            for m in sorted(Path(root).rglob("mask.npy")):
                folder = m.parent
                missing = [f for f in REQUIRED if not (folder / f"{f}.npy").exists()]
                if missing:
                    raise FileNotFoundError(f"{folder} missing {missing}")
                out.append(Sample(folder, str(event_of(folder))))
            if not out:
                raise FileNotFoundError(f"no Kuro Siwo samples under {root}")
            return out


        def event_from_info(folder: Path, key: str = "actid") -> str:
            """Default event resolver; `key` MUST be confirmed against a real info.json."""
            info = json.loads((folder / "info.json").read_text())
            if key not in info:
                raise KeyError(f"info.json in {folder} has no '{key}'; keys={list(info)}")
            return str(info[key])


        class KuroSiwoDataset(Dataset):
            """Returns x: float32 (7,224,224) normalised, y: int64 (224,224) with 255 = ignore."""

            def __init__(self, samples: Sequence[Sample], events: Sequence[str], mean: np.ndarray, std: np.ndarray,
                         pixel_size_m: float = 10.0, augment: bool = False, seed: int = 0):
                keep = set(map(str, events))
                self.samples = [s for s in samples if s.event_id in keep]
                if not self.samples:
                    raise ValueError("no samples left after event filtering")
                self.mean = np.asarray(mean, np.float32)[:, None, None]
                self.std = np.asarray(std, np.float32)[:, None, None]
                self.pixel = pixel_size_m
                self.augment = augment
                self.rng = np.random.default_rng(seed)

            def __len__(self) -> int:
                return len(self.samples)

            def load_raw(self, i: int) -> tuple[np.ndarray, np.ndarray]:
                p = self.samples[i].path
                ld = lambda n: np.load(p / f"{n}.npy").astype(np.float32)
                pre = np.stack([np.stack([ld("sec1_vv"), ld("sec1_vh")]), np.stack([ld("sec2_vv"),
ld("sec2_vh")])])
                post = np.stack([ld("flood_vv"), ld("flood_vh")])
                x = build_input_stack(pre, post, slope_degrees(ld("dem"), self.pixel))
                y = ld("mask").astype(np.int64)
                y[(ld("valid_mask") < 0.5) | ~np.isfinite(x).all(0)] = IGNORE_INDEX
                return x, y

            def __getitem__(self, i: int):
                x, y = self.load_raw(i)
                x = (np.nan_to_num(x) - self.mean) / (self.std + 1e-6)
                if self.augment:
                    # Radiometric only on SAR channels 0-5: global gain (dB) + small noise.
                    # NO geometric flips by default: they mirror look geometry (flips are an ablation).
                    gain = self.rng.uniform(-1.0, 1.0) / self.std[:4, 0, 0]
                    x[:4] += gain[:, None, None]
                    x[:6] += self.rng.normal(0, 0.05, size=x[:6].shape).astype(np.float32)
                return torch.from_numpy(x.astype(np.float32)), torch.from_numpy(y)


        def compute_channel_stats(ds: KuroSiwoDataset, max_samples: int = 2000, seed: int = 0) -> tuple[np.ndarray,
np.ndarray]:
            """Per-channel mean/std over TRAINING events only (valid pixels)."""
            rng = np.random.default_rng(seed)
            idx = rng.choice(len(ds), size=min(max_samples, len(ds)), replace=False)
            s = np.zeros(7); ss = np.zeros(7); n = 0
            for i in idx:
                x, y = ds.load_raw(int(i))
                v = y != IGNORE_INDEX
                xv = x[:, v].astype(np.float64)
                s += xv.sum(1); ss += (xv ** 2).sum(1); n += xv.shape[1]
            mean = s / n
```

```
            std = np.sqrt(np.maximum(ss / n - mean ** 2, 1e-12))
            return mean.astype(np.float32), std.astype(np.float32)
```

18.4 Model, checkpoint save/load — `ml/model.py`

File: `src/cutoff/ml/model.py`

```
        """U-Net (ResNet-34 encoder) for 7-channel same-track SAR change input -> 3 classes."""
        from __future__ import annotations

        import torch
        import torch.nn as nn

        N_CHANNELS = 7
        N_CLASSES = 3  # 0 no water, 1 permanent water, 2 flood (Kuro Siwo label scheme)


        def build_model(in_channels: int = N_CHANNELS, classes: int = N_CLASSES, encoder: str = "resnet34",
                        pretrained: bool = True) -> nn.Module:
            """segmentation_models_pytorch adapts the first conv to `in_channels` when
            in_channels != 3 (it re-uses/repeats the pretrained RGB kernels)."""
            import segmentation_models_pytorch as smp

            return smp.Unet(encoder_name=encoder, encoder_weights="imagenet" if pretrained else None,
                            in_channels=in_channels, classes=classes)


        def save_checkpoint(path, model: nn.Module, meta: dict) -> None:
            """Store weights + everything needed to reproduce preprocessing at inference."""
            required = {"channels", "mean", "std", "temperature", "flood_threshold", "config_hash", "train_events"}
            missing = required - set(meta)
            if missing:
                raise ValueError(f"checkpoint meta missing {missing}")
            torch.save({"state_dict": model.state_dict(), "meta": meta}, path)


        def load_checkpoint(path, device: str = "cpu") -> tuple[nn.Module, dict]:
            ck = torch.load(path, map_location=device, weights_only=False)
            if "state_dict" not in ck or "meta" not in ck:
                raise ValueError(f"{path} is not a Cutoff checkpoint")
            meta = ck["meta"]
            model = build_model(in_channels=len(meta["channels"]), pretrained=False)
            model.load_state_dict(ck["state_dict"])  # raises on shape mismatch -> invalid checkpoint
            return model.to(device).eval(), meta
```

18.5 Loss — `ml/losses.py`

File: `src/cutoff/ml/losses.py`

```
        """Class-weighted cross-entropy + soft Dice on the flood class."""
        from __future__ import annotations

        import torch
        import torch.nn as nn
        import torch.nn.functional as F

        IGNORE_INDEX = 255


        class CEDiceLoss(nn.Module):
            def __init__(self, class_weights=(1.0, 1.0, 3.0), dice_class: int = 2, w_ce: float = 1.0, w_dice: float
= 1.0):
                super().__init__()
                self.register_buffer("cw", torch.tensor(class_weights, dtype=torch.float32))
                self.dice_class, self.w_ce, self.w_dice = dice_class, w_ce, w_dice

            def forward(self, logits: torch.Tensor, target: torch.Tensor) -> torch.Tensor:
                # logits (B,3,H,W) float; target (B,H,W) int64 with 255 = ignore
                ce = F.cross_entropy(logits, target, weight=self.cw, ignore_index=IGNORE_INDEX)
                valid = (target != IGNORE_INDEX).float()
```

```
            p = logits.softmax(1)[:, self.dice_class] * valid
            g = (target == self.dice_class).float() * valid
            inter = (p * g).sum()
            dice = 1.0 - (2 * inter + 1.0) / (p.sum() + g.sum() + 1.0)
            return self.w_ce * ce + self.w_dice * dice
```

18.6 Training loop — `ml/train.py` and calibration — `ml/calibrate.py`

File: `src/cutoff/ml/train.py`

```
"""Training loop: event-level validation, early stopping on validation flood IoU, AMP on CUDA."""
from __future__ import annotations

import json
import logging
import random
import time
from pathlib import Path

import numpy as np
import torch
from torch.utils.data import DataLoader

from cutoff.ml.losses import CEDiceLoss
from cutoff.ml.metrics import ConfusionMeter

log = logging.getLogger("cutoff.train")


def seed_everything(seed: int) -> None:
    random.seed(seed); np.random.seed(seed); torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)


@torch.no_grad()
def evaluate(model, loader, device) -> tuple[dict, torch.Tensor, torch.Tensor]:
    """Returns flood scores plus raw logits/targets subsample (for temperature scaling)."""
    model.eval(); cm = ConfusionMeter(3); keep_l, keep_t = [], []
    for x, y in loader:
        logits = model(x.to(device)).float().cpu()
        cm.update(logits.argmax(1).numpy(), y.numpy())
        keep_l.append(logits[:, :, ::8, ::8]); keep_t.append(y[:, ::8, ::8])  # 1/64 pixel subsample
    return cm.scores(), torch.cat(keep_l), torch.cat(keep_t)


def train(model, train_ds, val_ds, out_dir: Path, epochs: int = 30, batch_size: int = 16, lr: float = 3e-4,
          weight_decay: float = 1e-4, patience: int = 6, num_workers: int = 2, seed: int = 42) -> Path:
    seed_everything(seed)
    out_dir = Path(out_dir); out_dir.mkdir(parents=True, exist_ok=True)
    device = "cuda" if torch.cuda.is_available() else "cpu"
    model.to(device)
    tl = DataLoader(train_ds, batch_size=batch_size, shuffle=True, num_workers=num_workers, drop_last=True)
    vl = DataLoader(val_ds, batch_size=batch_size, shuffle=False, num_workers=num_workers)
    opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=weight_decay)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs)
    loss_fn = CEDiceLoss().to(device)
    scaler = torch.amp.GradScaler("cuda", enabled=device == "cuda")
    best, bad, hist = -1.0, 0, []
    for ep in range(epochs):
        model.train(); t0 = time.time(); run = 0.0
        for x, y in tl:
            x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
            opt.zero_grad(set_to_none=True)
            with torch.autocast(device_type=device, enabled=device == "cuda"):
                loss = loss_fn(model(x), y)
            if not torch.isfinite(loss):
                raise FloatingPointError(f"non-finite loss at epoch {ep}; check normalisation/NaNs")
            scaler.scale(loss).backward(); scaler.step(opt); scaler.update()
            run += loss.item()
        sched.step()
        val, _, _ = evaluate(model, vl, device)
        rec = {"epoch": ep, "train_loss": run / max(len(tl), 1), "val": val, "sec": time.time() - t0}
        hist.append(rec); log.info(json.dumps(rec, default=float))
        torch.save(model.state_dict(), out_dir / "last.pt")
        score = val["iou"] if np.isfinite(val["iou"]) else -1
        if score > best:
```

```
                best, bad = score, 0
                torch.save(model.state_dict(), out_dir / "best.pt")
            else:
                bad += 1
                if bad >= patience:
                    log.info("early stop at epoch %d", ep); break
        (out_dir / "history.json").write_text(json.dumps(hist, default=float, indent=1))
        return out_dir / "best.pt"
```

File: `src/cutoff/ml/calibrate.py`

```
"""Temperature scaling (Guo et al., ICML 2017) fitted on VALIDATION events only."""
from __future__ import annotations

import torch
import torch.nn.functional as F

IGNORE_INDEX = 255


def fit_temperature(logits: torch.Tensor, target: torch.Tensor, max_iter: int = 200) -> float:
    """logits (N,C,H,W), target (N,H,W). Returns scalar T>0 minimising NLL. Argmax is unchanged."""
    z = logits.permute(0, 2, 3, 1).reshape(-1, logits.shape[1]).float()
    t = target.reshape(-1)
    keep = t != IGNORE_INDEX
    z, t = z[keep], t[keep]
    log_t = torch.zeros(1, requires_grad=True)
    opt = torch.optim.LBFGS([log_t], lr=0.1, max_iter=max_iter)

    def closure():
        opt.zero_grad()
        loss = F.cross_entropy(z / log_t.exp(), t)
        loss.backward()
        return loss

    opt.step(closure)
    return float(log_t.exp().item())
```

18.7 Metrics — `ml/metrics.py`

File: `src/cutoff/ml/metrics.py`

```
"""Confusion-matrix based segmentation metrics, accumulated per event / per stratum."""
from __future__ import annotations

from collections import defaultdict

import numpy as np

IGNORE_INDEX = 255


class ConfusionMeter:
    def __init__(self, n_classes: int = 3):
        self.n = n_classes
        self.cm = np.zeros((n_classes, n_classes), dtype=np.int64)  # rows = truth, cols = pred

    def update(self, pred: np.ndarray, target: np.ndarray, mask: np.ndarray | None = None) -> None:
        pred, target = np.asarray(pred).ravel(), np.asarray(target).ravel()
        v = target != IGNORE_INDEX
        if mask is not None:
            v &= np.asarray(mask).ravel().astype(bool)
        idx = target[v].astype(np.int64) * self.n + pred[v].astype(np.int64)
        self.cm += np.bincount(idx, minlength=self.n ** 2).reshape(self.n, self.n)

    def scores(self, cls: int = 2) -> dict:
        tp = self.cm[cls, cls]
        fp = self.cm[:, cls].sum() - tp
        fn = self.cm[cls, :].sum() - tp
        nan = float("nan")
        prec = tp / (tp + fp) if tp + fp else nan
        rec = tp / (tp + fn) if tp + fn else nan
```

```
                f1 = 2 * tp / (2 * tp + fp + fn) if (2 * tp + fp + fn) else nan
                iou = tp / (tp + fp + fn) if (tp + fp + fn) else nan
                ious = [self.cm[c, c] / (self.cm[c, :].sum() + self.cm[:, c].sum() - self.cm[c, c])
                        if (self.cm[c, :].sum() + self.cm[:, c].sum()) else nan for c in range(self.n)]
                return {"precision": prec, "recall": rec, "f1": f1, "iou": iou,
                        "miou": float(np.nanmean(ious)), "support_px": int(self.cm[cls, :].sum())}


        def slope_bin(slope_deg: np.ndarray, edges=(5.0, 15.0)) -> np.ndarray:
            """0: <5 deg, 1: 5-15 deg, 2: >=15 deg."""
            return np.digitize(slope_deg, edges).astype(np.int8)


        class StratifiedMeter:
            """Keeps a ConfusionMeter per (event, slope_bin) so pooled numbers can't hide terrain failures."""

            def __init__(self, n_classes: int = 3):
                self.m = defaultdict(lambda: ConfusionMeter(n_classes))

            def update(self, event: str, pred, target, slope_deg) -> None:
                b = slope_bin(slope_deg)
                for k in range(3):
                    self.m[(event, k)].update(pred, target, b == k)
                    self.m[("ALL", k)].update(pred, target, b == k)
                self.m[(event, "all")].update(pred, target)
                self.m[("ALL", "all")].update(pred, target)

            def table(self) -> list[dict]:
                return [{"event": e, "slope_bin": s, **cm.scores()} for (e, s), cm in sorted(self.m.items(),
key=str)]


        def stable_steep_fpr(pred_flood: np.ndarray, slope_deg: np.ndarray, hand_m: np.ndarray,
                             slope_min: float = 20.0, hand_min: float = 100.0) -> float:
            """Share of 'cannot-be-flooded' pixels (steep AND high above drainage) predicted as flood.
            Needs no reference map, so it can be computed for any judge-chosen AOI."""
            region = (slope_deg > slope_min) & (hand_m > hand_min)
            return float(pred_flood[region].mean()) if region.any() else float("nan")


        def expected_calibration_error(conf: np.ndarray, correct: np.ndarray, n_bins: int = 10) -> float:
            conf, correct = np.asarray(conf).ravel(), np.asarray(correct).ravel().astype(float)
            bins = np.minimum((conf * n_bins).astype(int), n_bins - 1)
            ece = 0.0
            for b in range(n_bins):
                s = bins == b
                if s.any():
                    ece += s.mean() * abs(correct[s].mean() - conf[s].mean())
            return float(ece)
```

18.8 Tiled inference on an unseen image — `ml/infer.py`

File: `src/cutoff/ml/infer.py`

```
        """Tiled inference over a full AOI raster with overlap blending (no seams, bounded memory)."""
        from __future__ import annotations

        import numpy as np
        import torch


        def _weights(tile: int) -> np.ndarray:
            w = np.hanning(tile + 2)[1:-1].astype(np.float32)
            return np.outer(w, w) + 1e-3


        @torch.no_grad()
        def predict_proba(model, x_norm: np.ndarray, tile: int = 224, overlap: int = 64, temperature: float = 1.0,
                          batch: int = 8, device: str = "cpu") -> np.ndarray:
            """x_norm: (7,H,W) normalised with TRAINING mean/std. Returns calibrated probs (3,H,W) float32.

            Pixels where any input channel is non-finite get probs = NaN (no valid observation).
            """
            if x_norm.ndim != 3:
                raise ValueError("x_norm must be (C,H,W)")
```

```
    C, H, W = x_norm.shape
    invalid = ~np.isfinite(x_norm).all(0)
    x = np.nan_to_num(x_norm, nan=0.0)
    ph, pw = max(0, tile - H), max(0, tile - W)
    x = np.pad(x, ((0, 0), (0, ph), (0, pw)), mode="reflect" if min(H, W) > 1 else "constant")
    Hp, Wp = x.shape[1:]
    step = tile - overlap
    ys = list(range(0, Hp - tile + 1, step)) + ([Hp - tile] if (Hp - tile) % step else [])
    xs = list(range(0, Wp - tile + 1, step)) + ([Wp - tile] if (Wp - tile) % step else [])
    acc = np.zeros((3, Hp, Wp), np.float32); wsum = np.zeros((Hp, Wp), np.float32); wt = _weights(tile)
    coords = [(r, c) for r in ys for c in xs]
    model.eval().to(device)
    for i in range(0, len(coords), batch):
        cb = coords[i:i + batch]
        xb = torch.from_numpy(np.stack([x[:, r:r + tile, c:c + tile] for r, c in cb])).to(device)
        pb = torch.softmax(model(xb).float() / temperature, 1).cpu().numpy()
        for (r, c), p in zip(cb, pb):
            acc[:, r:r + tile, c:c + tile] += p * wt; wsum[r:r + tile, c:c + tile] += wt
    probs = (acc / wsum)[:, :H, :W]
    probs[:, invalid] = np.nan
    return probs
```

18.9 Rule-based disturbance, gating and optical evidence

File: `src/cutoff/geo/change.py`

```
"""Scene-adaptive backscatter-disturbance detector (rule-based; covers what Kuro Siwo cannot label).

Kuro Siwo labels water only. Debris deposits and scoured channels often RAISE backscatter,
smooth wet mud LOWERS it. We flag statistically unusual same-track change inside the valley
corridor, with the threshold estimated from terrain that cannot have flooded in THIS scene,
instead of borrowing fixed dB limits from other events.
"""
from __future__ import annotations

import numpy as np


def robust_sigma(values: np.ndarray) -> float:
    v = values[np.isfinite(values)]
    if v.size < 100:
        raise ValueError("too few stable pixels to estimate noise; enlarge AOI or relax HAND_stable")
    return float(1.4826 * np.median(np.abs(v - np.median(v))))


def disturbance(d_db: np.ndarray, hand_m: np.ndarray, valid: np.ndarray, k: float = 3.0,
                hand_corridor_m: float = 60.0, hand_stable_m: float = 100.0) -> tuple[np.ndarray, dict]:
    """d_db: post-minus-pre change (dB) of one polarisation (VV recommended; VH reported too).

    Returns int8 map: 0 none, 1 decrease (water / smooth mud), 2 increase (roughness / debris),
    -1 invalid; plus a dict with the scene statistics (logged into results.json).
    """
    stable = valid & (hand_m > hand_stable_m)
    sig = robust_sigma(d_db[stable])
    med = float(np.nanmedian(d_db[stable]))
    z = (d_db - med) / max(sig, 1e-6)
    corridor = valid & (hand_m <= hand_corridor_m)
    out = np.zeros(d_db.shape, np.int8)
    out[corridor & (z < -k)] = 1
    out[corridor & (z > k)] = 2
    out[~valid] = -1
    return out, {"sigma_stable_db": sig, "median_stable_db": med, "k": k,
                 "hand_corridor_m": hand_corridor_m, "n_stable_px": int(stable.sum())}
```

File: `src/cutoff/geo/gating.py`

```
"""Physical terrain gating + evidence tiers. Downgrades, never silently deletes.

Output classes (uint8, written to hazard_class.tif):
  0 no hazard detected            1 flood water - high      2 flood water - possible
  3 water/smooth mud (rule only)  4 roughness/debris (rule) 5 permanent water
  254 no valid observation (shadow/layover/nodata)
```

```
        Evidence flags (uint8 bitfield): 1 model, 2 rule-decrease, 4 rule-increase, 8 S2-water-agree,
          16 S2-clear-no-water, 32 steep, 64 high-HAND
        """
        from __future__ import annotations

        import numpy as np

        NO_OBS = 254


        def gate(probs: np.ndarray, dist: np.ndarray, slope: np.ndarray, hand: np.ndarray, shadow_layover:
np.ndarray,
                 s2_water: np.ndarray | None = None, s2_clear: np.ndarray | None = None, p_high: float = 0.5,
                 slope_max: float = 15.0, hand_max: float = 30.0) -> tuple[np.ndarray, np.ndarray]:
            """probs (3,H,W) calibrated; dist from change.disturbance; s2_* optional bool maps (None = no S2)."""
            H, W = slope.shape
            cls = np.zeros((H, W), np.uint8); flags = np.zeros((H, W), np.uint8)
            p_flood = np.nan_to_num(probs[2], nan=0.0); p_perm = np.nan_to_num(probs[1], nan=0.0)
            model_flood = p_flood >= p_high
            steep, high = slope > slope_max, hand > hand_max
            flags[model_flood] |= 1; flags[dist == 1] |= 2; flags[dist == 2] |= 4
            flags[steep] |= 32; flags[high] |= 64
            cls[(dist == 1)] = 3
            cls[(dist == 2)] = 4
            cls[model_flood] = 1
            cls[model_flood & (steep | high)] = 2          # physically implausible -> "possible"
            cls[(p_perm >= p_high) & ~model_flood] = 5
            if s2_water is not None and s2_clear is not None:
                agree = s2_clear & s2_water; contra = s2_clear & ~s2_water
                flags[agree] |= 8; flags[contra] |= 16
                cls[(cls == 2) & agree] = 1                # optical corroboration can upgrade
                cls[(cls == 1) & contra & (dist != 1)] = 2  # clear-sky optical sees no water -> downgrade
            invalid = shadow_layover | ~np.isfinite(probs).all(0) | (dist == -1)
            cls[invalid] = NO_OBS
            return cls, flags
```

File: `src/cutoff/geo/optical.py`

```
        """Sentinel-2 L2A optical evidence (decision-level, optional).

        S2 is NOT a model input: no allowed training set provides labelled, temporally paired
        pre/post S1+S2 (Kuro Siwo is SAR-only; Sen1Floods11 is single-date). S2 is used as an
        independent, rule-based corroboration layer where the sky is clear.
        L2A bands: B03 green (10 m), B08 NIR (10 m), B11 SWIR1 (20 m -> resampled to 10 m), SCL (20 m).
        SCL classes treated as clear: 4 vegetation, 5 not-vegetated, 6 water, 7 unclassified, 11 snow.
        """
        from __future__ import annotations

        import numpy as np

        CLEAR_SCL = (4, 5, 6, 7, 11)


        def l2a_reflectance(dn: np.ndarray, offset: float = -1000.0, scale: float = 1e4) -> np.ndarray:
            """Processing baseline >= 04.00 adds BOA_ADD_OFFSET=-1000. Read the offset from product metadata."""
            return (dn.astype(np.float32) + offset) / scale


        def mndwi(green: np.ndarray, swir1: np.ndarray) -> np.ndarray:
            with np.errstate(divide="ignore", invalid="ignore"):
                return (green - swir1) / (green + swir1)


        def s2_evidence(green, swir1, scl, thr: float = 0.0) -> tuple[np.ndarray, np.ndarray]:
            """Returns (water_bool, clear_bool). Threshold 0.0 is the textbook MNDWI split - a starting
            value; turbid, sediment-laden water may need a lower value (test on validation scenes)."""
            clear = np.isin(scl, CLEAR_SCL)
            w = mndwi(green, swir1)
            return (w > thr) & clear, clear
```

18.10 Georeferenced outputs — `geo/rasterio_io.py`

File: `src/cutoff/geo/rasterio_io.py`

```
        """Georeferenced raster I/O, inspection, area and vectorisation. CRS is never dropped."""
        from __future__ import annotations

        from pathlib import Path

        import numpy as np
        import rasterio
        from rasterio.crs import CRS
        from rasterio.features import shapes


        def utm_crs_for(lon: float, lat: float) -> CRS:
            zone = int((lon + 180) // 6) + 1
            return CRS.from_epsg((32600 if lat >= 0 else 32700) + zone)  # Trishuli (85.3E, 28.1N) -> EPSG:32645


        def inspect(path: str | Path) -> dict:
            """Metadata check run on every input raster before use."""
            with rasterio.open(path) as ds:
                if ds.crs is None:
                    raise ValueError(f"{path}: no CRS - refusing to use an un-georeferenced raster")
                return {"path": str(path), "crs": ds.crs.to_string(), "res": ds.res, "shape": (ds.count, ds.height,
ds.width),
                        "dtype": ds.dtypes[0], "nodata": ds.nodata, "bounds": tuple(ds.bounds), "tags": ds.tags()}


        def write_geotiff(path, arr: np.ndarray, transform, crs, nodata=None, tags: dict | None = None) -> None:
            arr = arr if arr.ndim == 3 else arr[None]
            prof = {"driver": "GTiff", "height": arr.shape[1], "width": arr.shape[2], "count": arr.shape[0],
                    "dtype": arr.dtype, "crs": crs, "transform": transform, "nodata": nodata,
                    "compress": "deflate", "tiled": True, "blockxsize": 256, "blockysize": 256}
            with rasterio.open(path, "w", **prof) as dst:
                dst.write(arr)
                if tags:
                    dst.update_tags(**{k: str(v) for k, v in tags.items()})


        def class_areas_km2(cls: np.ndarray, transform, crs) -> dict[int, float]:
            """Area per class. Only valid in a projected, metre-based CRS (UTM); refuses geographic/Web
Mercator."""
            crs = CRS.from_user_input(crs)
            if not crs.is_projected or crs.to_epsg() == 3857 or crs.linear_units not in ("metre", "meter"):
                raise ValueError(f"area needs a metre-based equal-scale projection (UTM), got {crs}")
            px_km2 = abs(transform.a * transform.e) / 1e6
            vals, counts = np.unique(cls, return_counts=True)
            return {int(v): float(c * px_km2) for v, c in zip(vals, counts)}


        def vectorize(cls: np.ndarray, transform, crs, keep=(1, 2, 3, 4), min_px: int = 5, simplify_m: float =
5.0):
            """Class raster -> GeoDataFrame polygons (class, area_m2). Tiny specks (< min_px) dropped."""
            import geopandas as gpd
            from shapely.geometry import shape

            mask = np.isin(cls, keep)
            recs = [{"geometry": shape(g), "class": int(v)} for g, v in shapes(cls.astype(np.int16), mask=mask,
transform=transform)]
            gdf = gpd.GeoDataFrame(recs, geometry="geometry", crs=crs) if recs else gpd.GeoDataFrame(
                {"class": []}, geometry=[], crs=crs)
            if len(gdf):
                px = abs(transform.a * transform.e)
                gdf = gdf[gdf.area >= min_px * px].copy()
                gdf["geometry"] = gdf.geometry.simplify(simplify_m, preserve_topology=True)
                gdf["area_m2"] = gdf.area
            return gdf
```

18.11 Same-track pairing and OSM snapshot — `data/s1_pairs.py`, `data/osm.py`

File: `src/cutoff/data/s1_pairs.py`

```
        """Same-track Sentinel-1 pair selection from STAC item metadata (pure logic, unit-testable).

        Rule (PS tip): compare only acquisitions from the same relative orbit (same geometry, same
        time of day). Since mid-2026 Sentinel-1C and -1D fly the same tracks 6 days apart (CDSE news,
```

```
        'Sentinel-1 orbital reconfiguration dates'), so a same-track repeat can be 6 or 12 days.
        Platforms may differ; relative orbit + orbit direction may not.

        STAC property names used here follow the STAC SAR/sat extensions; CONFIRM against a real CDSE
        item in Phase 2 (`sat:relative_orbit`, `sat:orbit_state`, `sar:instrument_mode`,
        `sar:polarizations`) and adjust FIELD_MAP if CDSE differs.
        """
        from __future__ import annotations

        from collections import defaultdict
        from dataclasses import dataclass
        from datetime import datetime, timedelta

        FIELD_MAP = {"rel_orbit": "sat:relative_orbit", "direction": "sat:orbit_state", "mode":
"sar:instrument_mode",
                     "pols": "sar:polarizations"}


        @dataclass
        class Scene:
            id: str
            datetime: datetime
            rel_orbit: int
            direction: str
            coverage: float  # fraction of AOI covered by the footprint, 0-1
            pols: tuple[str, ...]


        class NoValidPair(Exception):
            pass


        def from_stac(item: dict, coverage: float) -> Scene:
            p = item["properties"]
            dt = datetime.fromisoformat(p["datetime"].replace("Z", "+00:00"))
            return Scene(item["id"], dt, int(p[FIELD_MAP["rel_orbit"]]), str(p[FIELD_MAP["direction"]]).lower(),
coverage,
                         tuple(x.upper() for x in p.get(FIELD_MAP["pols"], [])))


        def select_pairs(scenes: list[Scene], event: datetime, max_pre: int = 3, pre_window_days: int = 48,
                         post_window_days: int = 12, min_coverage: float = 0.8) -> list[dict]:
            """Return candidate (track -> 1 post + up to max_pre pre) sets, best first.

            Ranking: earliest post after the event (fresher), then coverage, then number of pre scenes.
            Raises NoValidPair with an explanation (and the next expected pass) if nothing qualifies.
            """
            ok = [s for s in scenes if s.coverage >= min_coverage and {"VV", "VH"} <= set(s.pols)]
            by_track = defaultdict(list)
            for s in ok:
                by_track[(s.rel_orbit, s.direction)].append(s)
            out = []
            for (orb, d), ss in by_track.items():
                post = sorted([s for s in ss if event <= s.datetime <= event + timedelta(days=post_window_days)],
                              key=lambda s: s.datetime)
                pre = sorted([s for s in ss if event - timedelta(days=pre_window_days) <= s.datetime < event],
                             key=lambda s: s.datetime, reverse=True)[:max_pre]
                if post and pre:
                    out.append({"rel_orbit": orb, "direction": d, "post": post[0], "pre": pre,
                                "post_lag_h": (post[0].datetime - event).total_seconds() / 3600,
                                "coverage": min([post[0].coverage] + [p.coverage for p in pre])})
            if not out:
                tracks = sorted({(s.rel_orbit, s.direction) for s in ok})
                raise NoValidPair(f"no same-track pre/post set within windows; tracks seen: {tracks}. "
                                  "Report 'no valid observation' - never fall back to cross-track silently.")
            return sorted(out, key=lambda r: (r["post_lag_h"], -r["coverage"], -len(r["pre"])))
```

File: `src/cutoff/data/osm.py`

```
        """Pre-event OpenStreetMap via the ohsome API (historical snapshot). Data-rule guard included."""
        from __future__ import annotations

        from datetime import date

        import requests

        OHSOME = "https://api.ohsome.org/v1/elements/geometry"
```

```
        EVENT_CUTOFF = date(2026, 8, 26)  # PS: OSM must be as it was BEFORE the event
        DEFAULT_SNAPSHOT = "2026-07-27"   # PS: latest available snapshot at time of writing

        FILTERS = {
            "roads": "highway in (motorway,trunk,primary,secondary,tertiary,unclassified,residential,track,"
                     "motorway_link,trunk_link,primary_link,secondary_link,tertiary_link) and geometry:line",
            "bridges": "bridge=yes and highway=* and geometry:line",
            "buildings": "building=* and geometry:polygon",
            "places": "place in (city,town,village,hamlet) and geometry:point",
            "health": "amenity in (hospital,clinic,doctors) or healthcare=*",
        }


        def assert_pre_event(snapshot: str, event_date: date) -> None:
            d = date.fromisoformat(snapshot)
            if d >= event_date:
                raise ValueError(f"OSM snapshot {snapshot} is not before event {event_date}: forbidden input (PS
data rules)")


        def tile_bbox(w: float, s: float, e: float, n: float, step: float = 0.1):
            """Split large AOIs into ~0.1 deg chunks to avoid ohsome timeouts."""
            x = w
            while x < e:
                y = s
                while y < n:
                    yield (x, y, min(x + step, e), min(y + step, n)); y += step
                x += step


        def fetch_layer(bbox: tuple, layer: str, snapshot: str, event_date: date, timeout: int = 120) -> dict:
            assert_pre_event(snapshot, event_date)
            feats = []
            for b in tile_bbox(*bbox):
                r = requests.post(OHSOME, data={"bboxes": ",".join(map(str, b)), "time": snapshot,
                                                "filter": FILTERS[layer], "properties": "tags"}, timeout=timeout)
                r.raise_for_status()
                feats += r.json().get("features", [])
            # de-duplicate features split across tiles by OSM id
            seen, uniq = set(), []
            for f in feats:
                k = f["properties"].get("@osmId")
                if k not in seen:
                    seen.add(k); uniq.append(f)
            return {"type": "FeatureCollection", "features": uniq,
                    "license": "ODbL 1.0 - (c) OpenStreetMap contributors", "snapshot": snapshot}
```

18.12 Road network and cut-off status — `geo/network.py`

File: `src/cutoff/geo/network.py`

```
        """Road graph, closure scenarios and settlement-access status.

        Correctness rules (from the audit, ISS-04):
          * OSM ways are noded at every shared vertex AND every line-line crossing before graph build
            (shapely.unary_union nodes lines), so a hazard touching one point closes ~one short edge,
            not a 20 km way.  Bridges (OSM layer/bridge tags) are kept on their own way so crossings
            with roads underneath are NOT noded - the caller passes them through `bridge` attribute.
          * A settlement with no baseline route is 'unknown', never 'cut_off'.
        """
        from __future__ import annotations

        import math

        import geopandas as gpd
        import networkx as nx
        import numpy as np
        from shapely.geometry import LineString, MultiLineString, Point
        from shapely.ops import substring, unary_union

        DEFAULT_SPEED_KMH = {"motorway": 60, "trunk": 50, "primary": 40, "secondary": 30, "tertiary": 25,
                             "unclassified": 20, "residential": 15, "track": 10, "path": 4, "footway": 4}


        def node_lines(roads: gpd.GeoDataFrame, max_seg_m: float = 100.0) -> gpd.GeoDataFrame:
```

```
    """Split roads at intersections. Grade-separated features (bridge=yes) are noded separately
    and only joined to the rest at their shared endpoints."""
    out = []
    is_bridge = roads.get("bridge", gpd.pd.Series(index=roads.index, dtype=object)).fillna("no").ne("no")
    for flag, part in ((False, roads[~is_bridge]), (True, roads[is_bridge])):
        if part.empty:
            continue
        merged = unary_union(list(part.geometry))
        segs = list(merged.geoms) if isinstance(merged, MultiLineString) else [merged]
        src = part.reset_index(drop=True)
        sidx = src.sindex
        for s in segs:
            cand = sidx.query(s, predicate="covers") if not s.is_empty else []
            if len(cand) == 0:
                cand = sidx.query(s.interpolate(0.5, normalized=True).buffer(0.01), predicate="intersects")
            hw = src.iloc[cand[0]]["highway"] if len(cand) else "unclassified"
            n = max(1, int(np.ceil(s.length / max_seg_m)))  # densify: settlements snap near their true
spot
            for k in range(n):
                out.append({"geometry": substring(s, k * s.length / n, (k + 1) * s.length / n),
                            "highway": hw, "bridge": flag})
    return gpd.GeoDataFrame(out, geometry="geometry", crs=roads.crs)


def build_graph(segments: gpd.GeoDataFrame, snap_m: float = 1.0) -> nx.Graph:
    """Undirected graph (one-way rules ignored: rescue access is two-way by assumption, stated)."""
    if not segments.crs or not segments.crs.is_projected:
        raise ValueError("segments must be in a projected metre CRS")
    G = nx.Graph()
    key = lambda xy: (round(xy[0] / snap_m) * snap_m, round(xy[1] / snap_m) * snap_m)
    for i, r in segments.iterrows():
        a, b = key(r.geometry.coords[0]), key(r.geometry.coords[-1])
        if a == b:
            continue
        v = DEFAULT_SPEED_KMH.get(str(r["highway"]), 15) / 3.6
        L = r.geometry.length
        if G.has_edge(a, b) and G[a][b]["length"] <= L:
            continue
        G.add_edge(a, b, seg=i, length=L, t=L / v, bridge=bool(r["bridge"]))
    return G


def closed_segments(segments: gpd.GeoDataFrame, hazard: gpd.GeoDataFrame, scenario: str) -> set[int]:
    """Scenario A conservative: segment touches any hazard (class 1-4) within 15 m.
       Scenario B moderate: >= 50 m of the segment inside HIGH-confidence hazard (class 1 or 4).
       Scenario C bridges only: bridge segment touches HIGH-confidence hazard within 15 m."""
    if hazard.empty:
        return set()
    high = hazard[hazard["class"].isin([1, 4])]
    if scenario == "A":
        h = unary_union(list(hazard.geometry)).buffer(15)
        return set(segments.index[segments.intersects(h)])
    if scenario == "B":
        if high.empty:
            return set()
        h = unary_union(list(high.geometry))
        return set(segments.index[segments.intersection(h).length >= 50])
    if scenario == "C":
        if high.empty:
            return set()
        h = unary_union(list(high.geometry)).buffer(15)
        return set(segments.index[segments["bridge"] & segments.intersects(h)])
    raise ValueError(f"unknown scenario {scenario}")


def snap_points(G: nx.Graph, pts: gpd.GeoDataFrame, max_m: float = 500.0) -> dict:
    """Map each point index to nearest graph node within max_m (else None -> 'unknown')."""
    nodes = np.array(list(G.nodes))
    out = {}
    for i, p in pts.geometry.items():
        if len(nodes) == 0:
            out[i] = None; continue
        d = np.hypot(nodes[:, 0] - p.x, nodes[:, 1] - p.y); j = int(d.argmin())
        out[i] = tuple(nodes[j]) if d[j] <= max_m else None
    return out


def access_status(G: nx.Graph, settlements: gpd.GeoDataFrame, targets: gpd.GeoDataFrame, closed: set[int],
                  degraded_ratio: float = 1.5) -> gpd.GeoDataFrame:
    s_node, t_node = snap_points(G, settlements), snap_points(G, targets)
```

```
            dests = {n for n in t_node.values() if n is not None}
            if not dests:
                raise ValueError("no hospital/town target snaps to the road graph")
            d0 = nx.multi_source_dijkstra_path_length(G, dests, weight="t")
            H = G.copy()
            H.remove_edges_from([(u, v) for u, v, d in G.edges(data=True) if d["seg"] in closed])
            d1 = nx.multi_source_dijkstra_path_length(H, dests, weight="t")
            rows = []
            for i, n in s_node.items():
                if n is None or n not in d0:
                    st, t0, t1 = "unknown", math.nan, math.nan
                else:
                    t0 = d0[n] / 60; t1 = d1.get(n, math.inf) / 60
                    st = "cut_off" if math.isinf(t1) else ("degraded" if t1 > degraded_ratio * max(t0, 1e-9) else
"connected")
                rows.append({"idx": i, "status": st, "t0_min": t0, "t1_min": t1})
            out = settlements.copy()
            out[["status", "t0_min", "t1_min"]] = gpd.pd.DataFrame(rows).set_index("idx")[["status", "t0_min",
"t1_min"]]
            return out


        def edge_dropout(G: nx.Graph, frac: float, rng: np.random.Generator) -> nx.Graph:
            """Simulate missing OSM roads for the sensitivity study (E6)."""
            H = G.copy(); edges = list(H.edges)
            k = int(round(frac * len(edges)))
            H.remove_edges_from([edges[i] for i in rng.choice(len(edges), size=k, replace=False)])
            return H
```

18.13 Results schema and situation report — `report/`

File: `src/cutoff/report/schema.py`

```
        """results.json: the single source of truth. Dashboard, sitrep and (optional) copilot read ONLY this."""
        from __future__ import annotations

        from datetime import date, datetime

        from pydantic import BaseModel, Field

        ATTRIBUTION = [
            "Contains modified Copernicus Sentinel data 2026.",
            "Produced using Copernicus WorldDEM-30 © DLR e.V. 2010–2014 and © Airbus Defence and Space GmbH "
            "2014–2018 provided under COPERNICUS by the European Union and ESA; all rights reserved.",
            "© OpenStreetMap contributors.",
            "Model trained on Kuro Siwo (Bountos et al., NeurIPS 2024), CC BY 4.0.",
        ]
        DISCLAIMER = "Educational prototype - not an operational tool. Exposure is not confirmed damage."


        class Inputs(BaseModel):
            aoi_wkt: str
            event_date: date
            osm_snapshot: date
            s1_post: str
            s1_pre: list[str]
            rel_orbit: int
            direction: str
            s2_scenes: list[str] = []
            s2_clear_fraction: float | None = None


        class Hazard(BaseModel):
            area_km2_by_class: dict[str, float]
            sigma_stable_db: float
            stable_steep_fpr: float | None
            no_observation_km2: float


        class Exposure(BaseModel):
            buildings_by_class: dict[str, int]
            road_km_by_class: dict[str, float]
            bridges_by_class: dict[str, int]
```

```
        class ScenarioResult(BaseModel):
            scenario: str
            closed_road_km: float
            counts: dict[str, int]          # connected / degraded / cut_off / unknown
            cut_off_names: list[str]


        class Results(BaseModel):
            run_id: str
            created_utc: datetime
            model_version: str
            inputs: Inputs
            hazard: Hazard
            exposure: Exposure
            access: list[ScenarioResult]
            limitations: list[str]
            attribution: list[str] = Field(default_factory=lambda: list(ATTRIBUTION))
            disclaimer: str = DISCLAIMER
```

File: `src/cutoff/report/sitrep.py`

```
        """One-page situation report rendered from results.json by a template. No language model: every
        number in the output is a field of `Results`, so nothing can be invented."""
        from __future__ import annotations

        from jinja2 import Environment, StrictUndefined

        from cutoff.report.schema import Results

        TEMPLATE = """<html><head><meta charset="utf-8"><style>
        body{font-family:sans-serif;font-size:10pt;margin:14mm} h1{font-size:14pt} table{border-collapse:collapse}
        td,th{border:1px solid #999;padding:2px 6px} .s{font-size:7.5pt;color:#444}</style></head><body>
        <h1>Situation report - flood/debris impact (run {{ r.run_id }})</h1>
        <p><b>{{ r.disclaimer }}</b></p>
        <p>Event date {{ r.inputs.event_date }}; Sentinel-1 post scene {{ r.inputs.s1_post }} (relative orbit
        {{ r.inputs.rel_orbit }}, {{ r.inputs.direction }}); {{ r.inputs.s1_pre|length }} same-track pre scenes;
        OSM snapshot {{ r.inputs.osm_snapshot }}.</p>
        <h2>Where did the flood hit?</h2><table><tr><th>Class</th><th>Area (km&sup2;)</th></tr>
        {% for k,v in r.hazard.area_km2_by_class.items() %}<tr><td>{{k}}</td><td>{{ '%.2f'|format(v) }}</td></tr>{%
endfor %}
        </table><p>No valid radar observation (shadow/layover/no data): {{
'%.2f'|format(r.hazard.no_observation_km2) }} km&sup2;.</p>
        <h2>What was exposed?</h2><table><tr><th>Class</th><th>Buildings</th><th>Road km</th><th>Bridges</th></tr>
        {% for k in r.exposure.buildings_by_class %}<tr><td>{{k}}</td><td>{{ r.exposure.buildings_by_class[k] }}
</td>
        <td>{{ '%.1f'|format(r.exposure.road_km_by_class.get(k,0)) }}</td><td>{{
r.exposure.bridges_by_class.get(k,0) }}</td></tr>{% endfor %}
        </table>
        <h2>Who is cut off? (range across closure assumptions)</h2><table><tr><th>Scenario</th><th>Cut off</th>
<th>Degraded</th>
        <th>Connected</th><th>Unknown (no mapped route)</th></tr>{% for s in r.access %}<tr><td>{{s.scenario}}</td>
        <td>{{s.counts.get('cut_off',0)}}</td><td>{{s.counts.get('degraded',0)}}</td><td>
{{s.counts.get('connected',0)}}</td>
        <td>{{s.counts.get('unknown',0)}}</td></tr>{% endfor %}</table>
        <h2>Limitations</h2><ul>{% for l in r.limitations %}<li>{{l}}</li>{% endfor %}</ul>
        <p class="s">{% for a in r.attribution %}{{a}}<br>{% endfor %}</p></body></html>"""


        def render_html(r: Results) -> str:
            return Environment(undefined=StrictUndefined, autoescape=True).from_string(TEMPLATE).render(r=r)
```

18.14 Serving and displaying a prediction — `app/streamlit_app.py`

For a hackathon, the “backend” is the CLI pipeline writing a run folder. Streamlit only *reads* that folder, which keeps one source of truth. If a REST API is later required, wrap `pipeline.run()` in a FastAPI `POST /runs` endpoint that returns `run_id`, plus `GET /runs/{id}/results.json`. The contracts above stay unchanged.

File: `app/streamlit_app.py`

```
        """Dashboard: reads a finished run folder (results.json + COG/GeoJSON layers). It never computes numbers
itself."""
        import json
        from pathlib import Path

        import folium
        import geopandas as gpd
        import streamlit as st
        from streamlit_folium import st_folium

        RUNS = Path("outputs")
        COLORS = {"cut_off": "#c0392b", "degraded": "#e67e22", "connected": "#27ae60", "unknown": "#7f8c8d"}

        st.set_page_config(page_title="Cutoff", layout="wide")
        st.sidebar.title("Cutoff")
        st.sidebar.caption("Educational prototype - not an operational tool.")
        with st.sidebar.form("run"):
            bbox = st.text_input("AOI bbox (W,S,E,N, EPSG:4326)", "85.25,28.05,85.40,28.20")
            event = st.date_input("Flood date")
            go = st.form_submit_button("Run pipeline")
        if go:
            st.sidebar.info("Run with: cutoff run --bbox ... --event-date ... (shows progress per stage); "
                            "live run is triggered via subprocess in the full app.")
        runs = sorted([p for p in RUNS.glob("*") if (p / "results.json").exists()], reverse=True)
        if not runs:
            st.warning("No finished runs in outputs/. Run the pipeline first."); st.stop()
        run = st.sidebar.selectbox("Run", runs, format_func=lambda p: p.name)
        r = json.loads((run / "results.json").read_text())
        scen = st.sidebar.radio("Closure scenario", [a["scenario"] for a in r["access"]], index=0)

        c1, c2 = st.columns([3, 2])
        with c1:
            hz = gpd.read_file(run / "hazard.gpkg").to_crs(4326)
            st_ = gpd.read_file(run / f"settlements_{scen}.gpkg").to_crs(4326)
            m = folium.Map(location=[hz.unary_union.centroid.y, hz.unary_union.centroid.x], zoom_start=12,
tiles="OpenStreetMap")
            folium.GeoJson(hz, name="Hazard", style_function=lambda f: {"color": "#1f4e79", "weight": 0.5,
                           "fillOpacity": 0.5 if f["properties"]["class"] == 1 else 0.25}).add_to(m)
            for _, s in st_.iterrows():
                folium.CircleMarker([s.geometry.y, s.geometry.x], radius=6, color=COLORS[s.status], fill=True,
                                    tooltip=f'{s.get("name", "?")}: {s.status}').add_to(m)
            folium.LayerControl().add_to(m)
            st_folium(m, height=620, use_container_width=True)
        with c2:
            i = r["inputs"]
            st.markdown(f"**Post scene** {i['s1_post']} · orbit {i['rel_orbit']} {i['direction']} · OSM
{i['osm_snapshot']}")
            st.metric("No valid radar observation (km²)", f"{r['hazard']['no_observation_km2']:.2f}")
            st.table(r["hazard"]["area_km2_by_class"])
            a = next(a for a in r["access"] if a["scenario"] == scen)
            st.table(a["counts"])
            st.download_button("Situation report (PDF)", (run / "sitrep.pdf").read_bytes(), "sitrep.pdf")
            st.download_button("Hazard polygons (GeoPackage, ODbL-free)", (run / "hazard.gpkg").read_bytes(),
"hazard.gpkg")
        st.caption(" · ".join(r["attribution"]))
```

18.15 Check-only evaluation — `eval/emsr927.py`

File: `eval/emsr927.py`

```
        """CHECK-ONLY comparison against Copernicus EMS EMSR927 reference layers.

        Lives outside src/ on purpose: nothing under src/cutoff may import this module or read
        reference/ (enforced by tests/test_no_reference_leak). Run only AFTER the pipeline has
        produced outputs/<run_id>/ with frozen parameters. Never tune thresholds on its output.
        Credit: "European Union, Copernicus Emergency Management Service data".
        """
        from __future__ import annotations

        import argparse
        import json
        from pathlib import Path

        import geopandas as gpd
```

```
        import numpy as np
        import rasterio
        from rasterio.features import rasterize

        from cutoff.ml.metrics import ConfusionMeter


        def rasterize_reference(ref: gpd.GeoDataFrame, like: Path) -> np.ndarray:
            """Burn reference polygons onto the prediction grid (same CRS/transform). 1 = hazard."""
            with rasterio.open(like) as ds:
                ref = ref.to_crs(ds.crs)
                return rasterize(((g, 1) for g in ref.geometry), out_shape=(ds.height, ds.width),
transform=ds.transform,
                                 fill=0, dtype="uint8")


        def boundary_f1(pred: np.ndarray, ref: np.ndarray, tol_px: int = 3) -> float:
            """Boundary F1 with a pixel tolerance (3 px = 30 m at 10 m) - fair to 10 m SAR vs VHR references."""
            from scipy.ndimage import binary_dilation, binary_erosion

            pb = pred & ~binary_erosion(pred); rb = ref & ~binary_erosion(ref)
            st = np.ones((2 * tol_px + 1,) * 2, bool)
            p = (pb & binary_dilation(rb, st)).sum() / max(pb.sum(), 1)
            r = (rb & binary_dilation(pb, st)).sum() / max(rb.sum(), 1)
            return float(2 * p * r / (p + r)) if p + r else float("nan")


        def main():
            ap = argparse.ArgumentParser()
            ap.add_argument("--run", required=True, type=Path)        # outputs/<run_id>
            ap.add_argument("--reference", required=True, type=Path)  # reference/EMSR927/<aoi>/*.gpkg (observed
event layer)
            ap.add_argument("--layer", default=None)
            a = ap.parse_args()
            cls_path = a.run / "hazard_class.tif"
            with rasterio.open(cls_path) as ds:
                cls = ds.read(1)
            ref = rasterize_reference(gpd.read_file(a.reference, layer=a.layer), cls_path).astype(bool)
            valid = cls != 254
            pred = np.isin(cls, [1, 3, 4])  # high-confidence water + rule-based change ("where did it hit")
            cm = ConfusionMeter(2); cm.update(pred[valid].astype(int), ref[valid].astype(int))
            out = {"pixel": cm.scores(cls=1), "boundary_f1_30m": boundary_f1(pred & valid, ref & valid),
                   "no_obs_fraction_of_reference": float((ref & ~valid).sum() / max(ref.sum(), 1))}
            (a.run / "emsr927_eval.json").write_text(json.dumps(out, indent=1, default=float))
            print(json.dumps(out, indent=1, default=float))


        if __name__ == "__main__":
            main()
```

18.16 Tests

File: `tests/test_core.py`

```
        """Synthetic-data tests: they run without any downloaded satellite data."""
        import ast
        import json
        from datetime import date, datetime, timedelta, timezone
        from pathlib import Path

        import geopandas as gpd
        import numpy as np
        import pytest
        import torch
        from rasterio.transform import from_origin
        from shapely.geometry import LineString, Point, box

        from cutoff.data.kuro_siwo import KuroSiwoDataset, index_samples
        from cutoff.data.osm import assert_pre_event
        from cutoff.data.s1_pairs import NoValidPair, Scene, select_pairs
        from cutoff.geo import network as net
        from cutoff.geo.change import disturbance
        from cutoff.geo.gating import NO_OBS, gate
        from cutoff.geo.rasterio_io import class_areas_km2, utm_crs_for, vectorize
        from cutoff.geo.sar import build_input_stack, to_db
```

```
        from cutoff.ml.calibrate import fit_temperature
        from cutoff.ml.infer import predict_proba
        from cutoff.ml.losses import CEDiceLoss
        from cutoff.ml.metrics import ConfusionMeter, stable_steep_fpr
        from cutoff.ml.model import build_model, load_checkpoint, save_checkpoint
        from cutoff.report.schema import ATTRIBUTION

        ROOT = Path(__file__).resolve().parents[1]


        def make_fake_kuro(root: Path, n=4, event="999"):
            rng = np.random.default_rng(0)
            for i in range(n):
                d = root / event / f"s{i}"; d.mkdir(parents=True)
                for k in ["flood_vv", "sec1_vv", "sec2_vv"]:
                    np.save(d / f"{k}.npy", rng.uniform(0.01, 0.3, (224, 224)).astype(np.float32))
                for k in ["flood_vh", "sec1_vh", "sec2_vh"]:
                    np.save(d / f"{k}.npy", rng.uniform(0.001, 0.08, (224, 224)).astype(np.float32))
                np.save(d / "dem.npy", rng.uniform(0, 50, (224, 224)).astype(np.float32))
                np.save(d / "mask.npy", rng.integers(0, 3, (224, 224)).astype(np.float32))
                np.save(d / "valid_mask.npy", np.ones((224, 224), np.float32))
                (d / "info.json").write_text(json.dumps({"actid": event}))


        def test_to_db_and_stack_shapes():
            assert to_db(np.array([1.0, 0.1, 0.0]))[1] == pytest.approx(-10.0)
            pre = np.full((2, 2, 8, 8), 0.1, np.float32); post = np.full((2, 8, 8), 0.01, np.float32)
            x = build_input_stack(pre, post, np.zeros((8, 8)))
            assert x.shape == (7, 8, 8) and np.allclose(x[4], -10.0)  # 0.01 vs 0.1 linear = -10 dB change


        def test_dataset_and_model_forward(tmp_path):
            make_fake_kuro(tmp_path)
            samples = index_samples(tmp_path, lambda p: p.parent.name)
            ds = KuroSiwoDataset(samples, ["999"], mean=np.zeros(7), std=np.ones(7), augment=True)
            x, y = ds[0]
            assert x.shape == (7, 224, 224) and x.dtype == torch.float32 and y.shape == (224, 224)
            m = build_model(pretrained=False)
            logits = m(x[None])
            assert logits.shape == (1, 3, 224, 224)
            loss = CEDiceLoss()(logits, y[None])
            assert torch.isfinite(loss)
            T = fit_temperature(logits.detach(), y[None])
            assert T > 0


        def test_checkpoint_roundtrip_and_invalid(tmp_path):
            m = build_model(pretrained=False)
            meta = {"channels": list(range(7)), "mean": [0] * 7, "std": [1] * 7, "temperature": 1.0,
                    "flood_threshold": 0.5, "config_hash": "x", "train_events": ["1"]}
            save_checkpoint(tmp_path / "m.pt", m, meta)
            m2, meta2 = load_checkpoint(tmp_path / "m.pt")
            assert meta2["temperature"] == 1.0
            torch.save({"foo": 1}, tmp_path / "bad.pt")
            with pytest.raises(ValueError):
                load_checkpoint(tmp_path / "bad.pt")


        def test_tiled_inference_shape_and_nodata():
            m = build_model(pretrained=False)
            x = np.random.default_rng(1).normal(size=(7, 300, 260)).astype(np.float32)
            x[:, :5, :5] = np.nan
            p = predict_proba(m, x, tile=224, overlap=64)
            assert p.shape == (3, 300, 260) and np.isnan(p[:, 0, 0]).all()
            assert np.allclose(np.nansum(p, 0)[10:, 10:], 1.0, atol=1e-4)


        def test_metrics():
            cm = ConfusionMeter(); t = np.array([2, 2, 0, 0, 255]); p = np.array([2, 0, 2, 0, 2])
            cm.update(p, t); s = cm.scores()
            assert s["iou"] == pytest.approx(1 / 3) and s["precision"] == 0.5 and s["recall"] == 0.5
            assert stable_steep_fpr(np.array([1, 0, 0, 0]), np.array([30, 30, 5, 30]), np.array([200, 200, 200,
5])) == 0.5


        def test_disturbance_and_gating():
            rng = np.random.default_rng(0)
            d = rng.normal(0, 1, (100, 100)).astype(np.float32)
            hand = np.full((100, 100), 200.0); hand[40:60] = 5  # valley corridor
            d[45:50, 10:20] = -8; d[52:55, 10:20] = +8
```

```
            out, stats = disturbance(d, hand, np.ones_like(d, bool))
            assert (out[45:50, 10:20] == 1).all() and (out[52:55, 10:20] == 2).all() and stats["sigma_stable_db"] <
1.5
            probs = np.zeros((3, 100, 100), np.float32); probs[0] = 1
            probs[:, 0:10, 0:10] = [[[0]], [[0]], [[1]]]
            slope = np.zeros((100, 100)); slope[0:5] = 40
            sl = np.zeros((100, 100), bool); sl[90:] = True
            cls, flags = gate(probs, out, slope, hand, sl)
            assert cls[7, 7] == 2           # model flood on high HAND -> downgraded to "possible"
            assert (cls[90:] == NO_OBS).all()


        def test_area_requires_utm():
            tr = from_origin(300000, 3110000, 10, 10)
            cls = np.ones((100, 100), np.uint8)
            assert class_areas_km2(cls, tr, "EPSG:32645")[1] == pytest.approx(1.0)
            with pytest.raises(ValueError):
                class_areas_km2(cls, tr, "EPSG:3857")
            assert utm_crs_for(85.3, 28.1).to_epsg() == 32645
            g = vectorize(cls, tr, "EPSG:32645", keep=(1,))
            assert len(g) == 1 and g.area.sum() == pytest.approx(1e6, rel=1e-3)


        def test_pair_selector_rejects_cross_track():
            ev = datetime(2026, 8, 26, 3, tzinfo=timezone.utc)
            S = lambda d, o, dr: Scene(f"{o}{dr}{d}", ev + timedelta(days=d), o, dr, 1.0, ("VV", "VH"))
            only_cross = [S(-6, 19, "ascending"), S(3, 121, "descending")]
            with pytest.raises(NoValidPair):
                select_pairs(only_cross, ev)
            ok = select_pairs(only_cross + [S(-9, 121, "descending"), S(-21, 121, "descending")], ev)
            assert ok[0]["rel_orbit"] == 121 and len(ok[0]["pre"]) == 2


        def test_osm_date_guard():
            assert_pre_event("2026-07-27", date(2026, 8, 26))
            with pytest.raises(ValueError):
                assert_pre_event("2026-08-27", date(2026, 8, 26))


        def _toy_network():
            crs = "EPSG:32645"
            # A long main road (one OSM way, 4 km) with a village at each end and a hospital in the middle,
            # plus a side road that crosses it. The hazard touches the main road only near x=3500.
            roads = gpd.GeoDataFrame({"highway": ["primary", "tertiary"], "bridge": ["no", "no"]},
                                     geometry=[LineString([(0, 0), (4000, 0)]), LineString([(1000, -500), (1000,
500)])], crs=crs)
            places = gpd.GeoDataFrame({"name": ["West", "East", "Orphan"]},
                                      geometry=[Point(0, 0), Point(4000, 0), Point(9000, 9000)], crs=crs)
            hosp = gpd.GeoDataFrame({"name": ["Hospital"]}, geometry=[Point(1000, 500)], crs=crs)
            hazard = gpd.GeoDataFrame({"class": [1]}, geometry=[box(3400, -20, 3600, 20)], crs=crs)
            return roads, places, hosp, hazard


        def test_noding_prevents_whole_way_closure_and_statuses():
            roads, places, hosp, hazard = _toy_network()
            seg = net.node_lines(roads)
            assert len(seg) == 40 + 10  # noded at the crossing, then densified to <=100 m pieces
            G = net.build_graph(seg)
            closed = net.closed_segments(seg, hazard, "A")
            assert len(closed) == 4  # 230 m hazard+buffer touches four 100 m pieces, not the 4 km way
            st = net.access_status(G, places, hosp, closed).set_index("name")["status"]
            assert st["West"] == "connected" and st["East"] == "cut_off" and st["Orphan"] == "unknown"


        def test_scenario_c_ignores_non_bridges():
            roads, places, hosp, hazard = _toy_network()
            seg = net.node_lines(roads)
            assert net.closed_segments(seg, hazard, "C") == set()


        def test_no_reference_leak():
            """Disqualification guard: pipeline code must never import eval/ or read reference/."""
            for f in (ROOT / "src").rglob("*.py"):
                src = f.read_text()
                tree = ast.parse(src)
                for node in ast.walk(tree):
                    if isinstance(node, (ast.Import, ast.ImportFrom)):
                        names = [a.name for a in node.names] + [getattr(node, "module", "") or ""]
                        assert not any(n.startswith("eval") for n in names), f"{f} imports eval"
                for banned in ["reference/", "EMSR", "emsr", "UNOSAT", "GEOID", "MMFlood", "ImpactMesh"]:
```

```
                assert banned not in src, f"{f} mentions {banned}"


    def test_attribution_strings_exact():
        assert ATTRIBUTION[0] == "Contains modified Copernicus Sentinel data 2026."
        assert "© OpenStreetMap contributors." in ATTRIBUTION
```

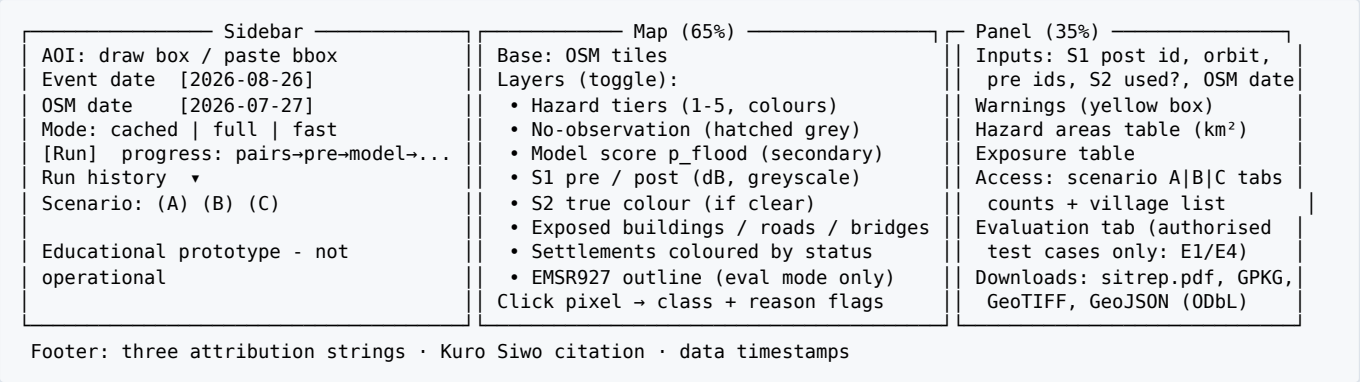
Integration tests to add in Phase 14:

- `test_pipeline_cached_trishuli`: end-to-end on cached rasters, asserting all artefacts exist and `results.json` validates;
- `test_two_aois`;
- `test_sitrep_pdf_has_attribution`: extract the PDF text and grep for the three strings;
- `test_geotiff_crs_preserved`;
- `test_no_s2_run`: S2 search mocked empty, the run must still succeed.

Logging requirement. Every stage logs JSON lines (`stage`, `seconds`, `peak_rss_mb`, input IDs, output paths, warnings) to `outputs/<run>/run.log`. Training logs each epoch's metrics to `history.json`.

19. Dashboard and demonstration workflow

19.1 Layout



19.2 Interactions, loading and error states

- **Run.** The sidebar launches `cutoff run` as a subprocess and polls the run log, showing the stage, elapsed time and the expected next stage. Cancel kills the subprocess.
- **States:**
  - *loading*: stage spinner;
  - *no valid pair*: red card with the tracks seen and the next pass;
  - *partial*: hazard OK but OSM failed, shown as an orange card with the hazard layers still displayed;
  - *success*.
- **Evaluation mode** appears only for runs whose ID matches a frozen evaluation run. It overlays EMSR927 outlines and shows the E4 table, so check-only data is never shown as an input.
- **Implemented vs proposed:** everything above except draw-to-select (use bbox text) and the EMSR927 overlay is in the starter app or MVP scope. Draw-to-select uses `streamlit-folium` draw events (Strong).

19.3 Situation report (one page)

The sitrep follows the template in §18.13:

- header with run ID, dates and a disclaimer;
- *Where:* areas by class plus the no-observation area;
- *What:* exposed buildings, road km and bridges by tier;
- *Who:* counts by scenario, and the names of cut-off villages in scenario B, with A and C as the range;
- limitations (5 bullets);
- attribution.

Every number is a `results.json` field. **Nepali version (Stretch):** a second template with fixed phrases translated and reviewed by a native speaker; numbers are inserted by the template, never generated.

19.4 Demo data needed

File	Source
`cache/trishuli/*`	S1 pre (2–3) + post, same track, preprocessed; GLO-30 mosaic; OSM 2026-07-27 layers
`cache/other_aoi/*`	Rehearsed non-Trishuli AOI with the same structure
`models/cutoff_unet_v1.pt`	Final checkpoint with meta
`reference/EMSR927/*`	EMS vectors (eval only)
`outputs/demo_mock/`	Synthetic run for UI fallback

19.5 3-minute video script

Time	Content
0:00–0:20	The rescuer’s question; one-line “educational prototype”; no victim imagery
0:20–1:15	Enter Trishuli AOI + date → same-track pair shown → hazard tiers, no-observation hatch, click a pixel to show the reason
1:15–1:50	Evaluation tab: Kuro Siwo held-out table (per slope bin) + EMSR927 comparison and disagreement taxonomy
1:50–2:30	Exposure + cut-off table; toggle scenarios A/B/C; explain “unknown”
2:30–2:50	Live run on a different valley (shows “works anywhere”)
2:50–3:00	Sitrep PDF, limitations in one line each, attributions

20. Testing and reliability

Case	What the system should do	Test
Corrupted / truncated download	Checksum or open fails → re-download once → cache fallback → clear error	mock
Missing files	Stage contract names the missing artefact	unit
Raster without CRS / wrong CRS	`inspect` refuses; reprojection at boundary; area refuses 3857	done (area)
Misaligned pre/post	NCC on stable terrain < threshold → warn; phase-correlation shift ≤ 2 px else abort	integration
No-data pixels	NaN → class 254, excluded from metrics, area reported	done
Cloudy optical	SCL clear fraction → S2 layer off	unit
Missing S2	Run succeeds without S2	integration
Wrong image dimensions	`build_input_stack` raises	done
Invalid checkpoint	`load_checkpoint` raises `ValueError`	done
Empty prediction	Valid; “no change detected” + no-obs area	unit
False detections (shadow, wet soil)	Gating downgrade; stable-steep FPR reported	done (gating)
Missed flood (narrow gorge, receded water)	Cannot fix fully; report recall by stratum; limitation stated	E2, E4
Geographic generalisation	E4 zero-tuning; domain-shift statement	E4
Inference latency	E7 table; cached/fast modes	E7
Memory limits	Windowed reads; tiled inference; SNAP subset	E7
Backend failure (stage crash)	Partial results with explicit warning	integration
Frontend failure	Static sitrep + GeoTIFF downloads still in run folder	manual
Rule violation	Leak test + OSM date guard in CI	done

Most serious risks:

1. live latency;
2. domain shift to the Himalaya;
3. false “cut off” results from OSM gaps;
4. no debris labels.

Mitigations are in §23.

Before any real-world use, the system would need validation across many Himalayan events with field-verified labels, a calibration study per region, a review by response practitioners of how tiers are interpreted, latency SLAs with a satellite tasking pipeline, and a formal error budget. None of this exists now, so **we do not describe the system as operationally reliable.**

21. Compute and cost analysis

All figures are **estimates with stated assumptions**, to be replaced by measurements in Phase 2 (storage) and E7 (runtime).

Item	Disk	RAM	GPU VRAM	Time	Assumption
Kuro Siwo labelled-GRD subset	30–60 GB (float32); ~half as float16	—	—	Download hours, network-bound	15k train + all val/test patches × 1.8 MB
Sentinel-1 GRD scenes (per AOI)	~1–2 GB per scene zip × 3–4	—	—	Minutes–hours per scene	Typical IW GRDH size **[U]**
SNAP Full preprocessing	~2–5 GB temp per scene	8–16 GB	—	8–15 min per full scene (audit, **[U]**); much less on an AOI subset	Subset node first
GLO-30 + terrain	< 1 GB	< 4 GB	—	< 1 min for 20 × 20 km	10 m grid of 2000² px
Training B3/P (subset)	checkpoints ~100 MB each	12 GB	≤ 8 GB fp16 at batch 16	~4–8 GPU-h incl. ablations **[U]**	T4
Training on CPU	—	16 GB	—	Days **[U]** — not planned	—
Inference 20 × 20 km	—	< 4 GB	optional	Minutes on CPU **[U]**	~100 tiles of 224² with overlap
Graph + exposure	—	< 2 GB	—	Seconds–minutes	< 100k edges
Dashboard	—	< 1 GB	—	Interactive	PNG overlays

Scenarios:

1. **CPU-only:** everything except main training is feasible. Use B0/B1 plus gating, and train only a small ResNet-18 model on a reduced subset if needed.
2. **Free GPU when available:** the plan above. Free quotas change and are not guaranteed, so keep checkpoints on Drive.
3. **Paid compute:** only justified if free GPU disappears during D5–8. A few hours of a single cloud GPU or Colab Pro for one month is the order of tens of US dollars **[U]**. It isn’t needed for the MVP.

Needs no GPU: data, preprocessing, terrain, B0, gating, exposure, graph, dashboard and report. **Benefits from a GPU:** U-Net training and ablations.

22. Hackathon execution plan

22.1 Judge’s-eye matrix

Dimension	Judges look for	Evidence we need	Proving deliverable	Likely weakness	Quick fix
Problem relevance	Answers the 3 questions	All three in one run	Dashboard	Debris only rule-based	Label it honestly; show both signs
PS compliance	Data rules, attribution	Leak test, date guard	CI log, ATTRIBUTION.md	Forgotten footer	Attribution test
Technical depth	Not a toy classifier	Same-track, gating, graph	Code + report §3	Over-scoping	MVP first
Research foundation	Grounded choices	Citations, conflict table	Report	AI-sourced claims	[V]/[S]/[U] labels
Defensible novelty	Honest contribution	E2, E4, E6	Tables	Claims too broad	Engineering + evaluation only
Data quality	Clean splits	Event split, QA plots	`docs/data_inventory.md`	Unknown `info.json`	Phase 2 check
Model evaluation	Held-out, stratified	E1, E2, E8	Tables	Few seeds	≥ 1 seed, 3 if time
Multimodal contribution	S1 + S2 used sensibly	E9 tier change	Map layer + table	S2 cloudy	Show clear-sky subset; explain fallback
Real-world usability	Actionable output	Sitrep, statuses, warnings	PDF + UI	Clutter	Default to scenario B + range
Reproducibility	Runs for others	Fresh clone	README, env lock	Hidden paths	Teammate test
Working demo	Live on judge AOI	E7	Video + live	Latency	Cache + fast mode + small AOI
Feasibility	Finished	All MVP ticks	Checklist	Integration late	Mocks from D6
Presentation	Clear, honest	Limitations	Report, video	Too technical	§2-style explanation

22.2 Recommendations

- **Three strongest differentiators:** (1) same-track, terrain-gated mapping with slope-stratified held-out metrics; (2) zero-tuning Trishuli test with a disagreement taxonomy; (3) range-based cut-off analysis with “unknown”.
- **Minimum credible baseline:** B0 robust change threshold with permanent-water and HAND masks, on the same events.
- **Highest-value experiment: E2** (gating by slope stratum). It is cheap, directly about mountain terrain, and needs no extra labels.

- **Most compelling demo:** a live run on a valley the team did not prepare. If it answers honestly with “no same-track post scene yet”, that is also a strong moment.
- **Evidence behind every performance claim:**

Claim	Required evidence
IoU / F1	E1/E4 tables with split and model version
“Gating reduces false alarms”	E2
“Same-track matters”	E5
“Cut-off robust / sensitive”	E6
“Calibrated”	E8
“Fast enough”	E7

- **Why this is more than a satellite-image classifier:** it selects physically comparable observations, encodes radar geometry and hydrology as explicit constraints, separates exposure from damage, turns maps into network reachability under declared assumptions, and reports what it could not observe.

22.3 Questions for the organisers (ask on day 1)

1. In the live round, may we use pre-downloaded or cached scenes, and how long is the slot? Is internet available for CDSE?
2. Does running our own preprocessing on CDSE GRD satisfy “raw satellite data”, and would a CDSE-hosted processed S1 product (if any) also count?
3. May sub-metre optical images (CC BY-NC) be shown for visual context only, or should they be avoided?
4. Under the segmentation option, is a Nepali sitrep expected?
5. Is the SRTM → Copernicus DEM substitution in our preprocessing acceptable, given that SRTM is not on the allowed list? (We already avoid SRTM; this question confirms that Kuro Siwo’s own SRTM use is not an issue.)

23. Risks and limitations

23.1 Risk register

Risk	L	I	Mitigation	Fallback preserving the PS
Live preprocessing too slow	H	H	Subset first; cache; fast mode; timing tests	Cached Trishuli + small live AOI
Kuro Siwo download too large/slow	M	H	Per-event `allow_patterns`; float16	Fewer events; B1 weights
No GPU	M	M	Colab/Kaggle; ResNet-18	B0/B1 + gating
Domain shift to Himalaya	H	H	Gating, slope channel, adaptive thresholds	Report measured transfer gap
Debris not labelled	H	M	Rule layer, clearly labelled	Water-only claim
OSM gaps → false cut-off	H	M	“Unknown” status; dropout study	Ranges, not single numbers
No same-track post scene for judge AOI	M	H	Explicit message + next pass	Cross-track ablation, watermarked
CDSE outage	M	H	Pre-downloaded scenes	Cached runs
SRTM→GLO-30 shift	M	L	E3b	Note in limitations
Leakage / rule violation	L	Critical	Folder separation, CI tests	Remove dependency before submission
Integration late	M	H	Mock `results.json` from D6	CLI + static sitrep

23.2 Limitations to state in the report

- **No early warning.** The revisit is 6–12 days on a given track; the system maps after the fact and may miss the flood peak.
- **10 m pixels.** Narrow channels, single roads and small bridges are below or near resolution, and road/bridge closure is inferred from proximity.
- **Radar geometry.** Shadow/layover areas are unobservable from one track and are shown as such.
- **Smooth wet mud can look like water**, and rough debris is only detected as statistical change, not classified.
- **Exposure ≠ damage.** No structural assessment is made.
- **The model score is calibrated on global, mostly lowland floods** and is not a Himalayan probability.

- **OSM completeness drives the access results.** “Unknown” means unmapped, not safe.
- **EMSR927 is an analyst VHR product**, not ground truth. Differences can be definitional or temporal.
- **Educational prototype only.**

24. Future improvements

- Conformal prediction (class-conditional) for an abstain class, calibrated on held-out *events*.
- SLC coherence loss for urban and debris areas (Kuro Siwo also provides SLC).
- Multi-track fusion: combine ascending and descending decisions at the polygon level (not pixel differencing) to reduce shadow gaps.
- Learned debris class from new expert labels on Himalayan events, if they become available with compatible licences.
- Population-weighted access, if the organisers allow an open population layer as input in a future edition.
- Reviewed bilingual (English/Nepali) sitrep; an optional grounded copilot that may only quote `results.json` fields.
- Docker image and a hosted demo.

25. Final conclusions

The PS is answerable within 15 days with free data, provided the team builds the simplest *correct* thing:

- same-track Sentinel-1 change with a modest U-Net trained only on Kuro Siwo;
- physical gating;
- a properly noded road graph;
- a template report fed by one JSON file.

Verification changed several assumptions in the earlier plans: dataset size, preprocessing radiometry, DEM compliance, revisit, and the role of Sentinel-2. Those changes are now built into the design and the starter code. The project's credibility will come from **evidence and honesty**: event-held-out and slope-stratified metrics, a zero-tuning Himalayan test, explicit "no observation" and "unknown" states, and ranges instead of single numbers. It will not come from claims of novelty or accuracy made before the experiments exist.

26. References

Official and primary (verified 11 Oct 2026 [V])

1. Multimodal AI Hackathon 2026, *Track B: Mapping Flood Damage from Space* (supplied PDFs `Track_B.pdf`, `Space_Track.pdf`).
2. Bountos N.I. et al. (2024). *Kuro Siwo: 33 billion m² under the water. A global multi-temporal satellite dataset for rapid flood mapping.* NeurIPS 37, 38105–38121. https://arxiv.org/abs/2311.12056
3. Kuro Siwo code: https://github.com/Orion-AI-Lab/KuroSiwo (MIT) — files `configs/grd_preprocessing.xml`, `configs/train/data_config.json`, `README.md`, `requirements.txt`.
4. Kuro Siwo data: https://huggingface.co/datasets/orion-ai-lab/Kuro-Siwo-GeoTIFFs and https://huggingface.co/datasets/orion-ai-lab/Kuro-Siwo-Webdataset (CC BY 4.0).
5. Copernicus EMS, *Flood in Nepal (EMSR927)*: https://mapping.emergency.copernicus.eu/news/flood-in-nepal-emsr927/ — credit “European Union, Copernicus Emergency Management Service data”.
6. CDSE Sentinel-1 documentation: https://documentation.dataspace.copernicus.eu/Data/SentinelMissions/Sentinel1.html
7. CDSE news, *Sentinel-1 orbital reconfiguration dates*: https://dataspace.copernicus.eu/node/3949
8. Microsoft Planetary Computer, *Sentinel-1 RTC* collection: https://planetarycomputer.microsoft.com/api/stac/v1/collections/sentinel-1-rtc
9. pyflwdir API reference: https://deltares.github.io/pyflwdir/latest/reference.html
10. ohsome API: https://docs.ohsome.org/ohsome-api/stable/ ; endpoint https://api.ohsome.org/v1
11. GEOID-Flood (2026): https://arxiv.org/abs/2608.02315 ; MMFlood: https://github.com/edornd/mmflood
12. EU Space “Image of the day” pages on the Nepal flood: https://eu-space.europa.eu/components/earth-observation-copernicus/image-of-the-day/aftermath-nepal-flash-flood

From supplied research, not re-verified [S]

13. Bonafilia D. et al. (2020). Sen1Floods11. CVPRW. doi:10.1109/CVPRW50498.2020.00113 — https://github.com/cloudtostreet/Sen1Floods11
14. Helleis M. et al. (2022). Sentinel-1-based water and flood mapping: benchmarking CNNs against an operational rule-based processing chain. IEEE JSTARS. doi:10.1109/jstars.2022.3152127

15. Bai Y. et al. (2021). Fusing Sentinel-1 and Sentinel-2 … Sen1Floods11. Remote Sensing 13(11):2220. doi:10.3390/rs13112220
16. Yang Z. et al. (2021). Mapping outburst floods … Baige landslide dam. Remote Sensing 13(11):2205. doi:10.3390/rs13112205
17. Petricola S. et al. (2022). Road criticality and loss of healthcare accessibility during floods: Cyclone Idai. Int. J. Health Geographics. doi:10.1186/s12942-022-00315-2
18. Kopiika N. et al. (2024). Rapid post-disaster infrastructure damage characterisation. Automation in Construction. doi:10.1016/j.autcon.2024.105955
19. Warnier M. et al. (2020). Humanitarian access, interrupted. OR Spectrum. doi:10.1007/s00291-020-00582-0
20. Kwant M. et al. (2021). Near real-time flood impact analysis on road networks. EGU21. doi:10.5194/egusphere-egu21-16545
21. Turkmenli I. et al. (2025). Domain generalization for SAR flood segmentation. JURSE. doi:10.1109/JURSE60372.2025.11075977
22. Zhao J. et al. (2024). UrbanSARFloods. CVPRW. doi:10.1109/CVPRW63382.2024.00047
23. Gupta A. et al. (2019). Post disaster mapping with semantic change detection. CVPRW. doi:10.1109/CVPRW.2019.00062
24. Hümmer C. et al. (2024). Uncertainty-aware 2D/3D change detection. IGARSS. doi:10.1109/IGARSS53475.2024.10642271
25. Kedia T. et al. (2020). Technologies enabling situational awareness during disaster response. DMPHP. doi:10.1017/dmp.2020.196
26. Tsutsui K. et al. (2004). Data fusion for debris hazard assessments. IGARSS. doi:10.1109/IGARSS.2004.1369035
27. Guo C. et al. (2017). On calibration of modern neural networks. ICML.
28. Nobre A.D. et al. (2011). HAND, a new terrain descriptor. J. Hydrology (HAND concept).
29. Repositories: github.com/JakubNiedzwiedz/SAR-Flood-Detection; github.com/AliForghani/sentinel1-sar-flood-mapping; github.com/GIScience/healthcare-access-idai; github.com/gboeing/osmnx; github.com/cordmaur/Sentinel1-Flood-Finder (GPL-3.0).
30. Supplied documents: *Cutoff — Track B project blueprint* (10 Oct 2026); *Independent … Audit: Project "Cutoff"*; *Technical Feasibility and Architectural Assessment for Space-Based Alpine Flood and Debris Mapping*; *Flood Mapping & Geospatial AI — Repository Investigation Report*; Elicit review and sources sheet.

Required attribution (verbatim from the PS): "Contains modified Copernicus Sentinel data 2026." · "Produced using Copernicus WorldDEM-30 © DLR e.V. 2010–2014 and © Airbus Defence and Space GmbH 2014–2018 provided under COPERNICUS by the European Union and ESA; all rights reserved." · "© OpenStreetMap contributors." · plus the Kuro Siwo citation (ref. 2).

27. Appendices

27.1 `configs/default.yaml` (template)

```yaml
run:
  utm_from_aoi: true            # EPSG chosen from AOI centroid (Trishuli -> 32645)
  pixel_m: 10
  cache_dir: cache/
osm:
  snapshot: "2026-07-27"        # must be < event date (asserted)
  tile_deg: 0.1
s1:
  pre_window_days: 48
  post_window_days: 12
  max_pre: 3
  min_coverage: 0.8
  require_pols: [VV, VH]
  mode: cached                  # cached | full | fast
model:
  checkpoint: models/cutoff_unet_v1.pt   # contains mean/std, T, tau, channels
  tile: 224
  overlap: 64
rules:
  k_sigma: 3.0                  # fixed before EMSR927
  hand_corridor_m: 60
  hand_stable_m: 100
gating:
  p_high: 0.5                   # replaced by tau from checkpoint
  slope_max_deg: 15
  hand_max_m: 30
s2:
  enabled: true
  max_days_from_s1: 5
  min_clear_fraction: 0.10
  mndwi_threshold: 0.0
access:
  scenarios: [A, B, C]
  degraded_ratio: 1.5
  snap_max_m: 500
  max_segment_m: 100
  targets: [hospital, town]
```

27.2 `configs/kuro_siwo_splits.yaml` (from the repo's `data_config.json` [V])

```yaml
source: Orion-AI-Lab/KuroSiwo configs/train/data_config.json (verified 2026-10-11)
train: [130, 470, 555, 118, 174, 324, 421, 554, 427, 518, 502, 498, 497, 496, 492, 147, 267, 273,
        275, 417, 567, 1111011, 1111004, 1111009, 1111010, 1111006, 1111005]
val:   [514, 559, 279, 520, 437, 1111003, 1111008]
test:  [321, 561, 445, 562, 411, 1111002, 277, 1111007, 205, 1111013]
exclude_within_km_of: {lat: 28.1, lon: 85.3, km: 100}   # check with catalogue.gpkg in Phase 2
```

27.3 Experiment tracking and reproducibility

- **Configuration:** YAML files in `configs/`. Each run writes `config_resolved.yaml` and its SHA-256 (`config_hash`) into the run folder and the checkpoint meta.
- **Seeds:** `seed_everything(seed)` for Python, NumPy and torch; `torch.backends.cudnn.benchmark=False` for final runs.
- **Dependencies:** `environment.yml` + `conda list --explicit > env.lock`; the Colab cell pins the same versions.
- **Dataset versioning:**

    - `data/kuro_index.csv` (sample path, event, split, flood fraction) with its hash;
    - the HF revision / commit used;
    - Trishuli product IDs in `pairs.json`.
- **Checkpoint naming:** `cutoff_unet_{inputs}_{encoder}_s{seed}_{config_hash[:8]}_{date}.pt`, e.g. `cutoff_unet_7ch_r34_s42_1a2b3c4d_20261018.pt`.
- **Logs:**
    - training: `history.json` (per-epoch loss, validation P/R/F1/IoU, seconds);
    - pipeline: `run.log` JSON lines.
- **Metrics recording:** `eval/*.py` write `metrics_{exp}.json` and CSV tables, each containing `model_version`, `config_hash`, `split`, `git_commit`.
- **Hardware/runtime:** GPU name, CUDA, CPU, RAM and wall time are recorded in each metrics file.
- **Reproduction:**

```
git clone <repo> && cd cutoff && conda env create -f environment.yml && conda activate cutoff
make test
python scripts/fetch_kuro_subset.py --config configs/default.yaml --index data/kuro_index.csv
cutoff train --config configs/train_unet.yaml --seed 42
cutoff eval-kuro --checkpoint models/<name>.pt --strata slope
cutoff run --bbox 85.25,28.05,85.40,28.20 --event-date 2026-08-26 --mode cached
python eval/emsr927.py --run outputs/<run_id> --reference reference/EMSR927/<aoi>.gpkg
streamlit run app/streamlit_app.py
```

Experiment log template (`docs/experiment_log.md`):

Date	Exp	Hypothesis	git commit	config_hash	Data split / index hash	Seeds	Hardware	Result file	Observation	Decision

Model evaluation report template:

```
Model: <checkpoint name>   Trained on: Kuro Siwo train events (27), excluded: <list>
Inputs: 7ch [VVpre, VHpre, VVpost, VHpost, dVV, dVH, slope]  Normalisation: train-event stats (in meta)
Threshold tau: <x> (chosen on val events)  Temperature T: <x> (val events)  ECE before/after: <x>/<x>
Held-out test (10 events): per-event table + pooled; slope-stratified table
Himalayan zero-tuning (Trishuli/EMSR927): table + disagreement taxonomy + no-observation share
Known failure modes (gallery): ...
Not claimed: damage, depth, operational reliability
```

27.4 Prioritised checklist

Must complete before coding

- [ ] Freeze the requirements matrix (§3.3); send the organiser questions (§22.3)
- [ ] CDSE accounts; confirm the Kuro Siwo layout, `info.json` keys and units; download `catalogue.gpkg` and check the 100 km exclusion
- [ ] Confirm the Trishuli same-track pre/post availability (relative orbits) via STAC
- [ ] Agree the `results.json` schema and stage contracts (§14.3)

Must complete for the MVP

- [ ] Pair selector + preprocessing (cached + full) for Trishuli and one other AOI
- [ ] B0 baseline; U-Net B3/P trained; τ and T fixed on validation events
- [ ] Disturbance + gating; vectorise + areas in UTM; exposure
- [ ] Graph with noding/densify; scenarios A and B; four statuses
- [ ] `results.json`, dashboard, sitrep PDF with attribution
- [ ] E1, E4, E6(i, iii) tables; leak test and date guard in CI; README fresh-clone run

Must complete before the final demo

- [ ] `git tag frozen-v1` before the first EMSR927 evaluation; E4 taxonomy written
- [ ] E7 timing on 10/20/40 km AOIs; rehearsed live run on an unprepared AOI
- [ ] Report ≤ 6 pages with limitations; video ≤ 3:00; attributions everywhere; no victim imagery

Optional post-hackathon

- [ ] E2, E3, E5, E8, E9 (if not done); scenario C + dropout; S2 evidence; bonus flow path
- [ ] Conformal abstain; SLC coherence; Nepali template; Docker; more Himalayan check events

27.5 Final validation audit

Audit item	Status	Evidence	Remaining action
Every feature satisfies the PS or has a justified purpose	Verified	§3.3 matrix; additions table	—
Selected dataset supports the task	Verified	Kuro Siwo has pre/post S1 + flood labels (HF card, data_config)	—
Dataset provides the modalities the model expects	Verified	VV/VH × 3 dates + DEM per sample; slope derived	Confirm units on real files (Phase 2)
Architecture, code and tensor shapes agree	Verified (synthetic)	`test_dataset_and_model_forward`, `test_tiled_inference...`	Re-run on real patches
Training and inference use consistent preprocessing	Partially verified	Shared `build_input_stack`; SNAP graph mirrors Kuro Siwo except DEM/projection	E3b; Fast-mode consistency check
Evaluation splits avoid leakage	Partially verified	Official event split; Trishuli unseen; leak test	Confirm 100 km exclusion with `catalogue.gpkg`
Reported metrics not confused with targets	Verified	No results reported; [TARGET] labels	Keep labels in report
Tech stack internally compatible	Partially verified	torch + smp + rasterio + geopandas + networkx + pyflwdir installed together and tested	SNAP version; WeasyPrint system libs
Every major file has a clear responsibility	Verified	§16.1	—

Audit item	Status	Evidence	Remaining action
MVP achievable in the stated time	Partially verified	Critical path ≈ 11 serial days; mocks from D6	Track daily; cut Strong items first
Demo uses available, legal data	Partially verified	Copernicus open data; OSM ODbL; Kuro Siwo CC BY	Trishuli scene availability (Phase 2)
Licensing and attribution addressed	Verified	ATTRIBUTION strings tested; ODbL notice on OSM exports	Footer on PDF test
Geospatial outputs preserve coordinates	Verified (unit)	`write_geotiff` profile; area refuses 3857; vectorise keeps CRS	Integration test on real run
Missing data and uncertainty communicated	Verified (design + unit)	Class 254; “unknown” status; S2-missing path; tiers	UI wording review
No unverified claim presented as a result	Verified	Labels [V]/[S]/[U]/[TARGET] throughout	Re-audit the final report
Citations and links traceable	Verified	§26 with URLs/DOIs	—
Compliance with track and submission rules	Partially verified	R1–R19 mapped; leak test	Organiser answers (§22.3)